nanuq

MANUAL TÉCNICO

Cómo está hecho Nanuq

Arquitectura con diagramas, cada plataforma por dentro y, al final, cada función del código.





Versión 1.46.1 · 9 de noviembre de 2026 · Español · Software libre (GPL-3.0-or-later)



Índice

La parte 1 explica el diseño; la parte 2 es la referencia de cada función.

ARQUITECTURA

1 **Panorama** 6
Una sola interfaz, tres envolturas.

2 **El repositorio** 8
Qué hay en cada carpeta.

3 **La página** 9
Cómo arranca, navega y reacciona app.js.

4 **Datos y almacenamiento** 11
Lo que dice algo de ti va cifrado.

5 **Reproducción** 13
De dónde sale cada canción y qué pasa cuando algo falla.

6 **Biblioteca y Jellyfin** 15
La API que usa Nanuq y lo que arma por su cuenta.

7 **DJ Nanuq** 16
Rondas con reglas, sin IA.

8 **Mis dispositivos** 17
Un Nanuq suena y los demás lo controlan, a través de Jellyfin.

9 **La fiesta** 18
Teléfonos que mandan canciones a una computadora por la red local.

10 **La app de Android** 20
WebView + Java nativo.

11 **La app de escritorio** 21
Electron para Linux y Windows.

12 **Seguridad** 22
Lo que llega de afuera nunca se cree.

13 **Compilar y publicar** 23
Una orden para todas las plataformas.

14 **Convenciones y trampas** 24
Lo que conviene saber antes de cambiar algo.

15 **Licencia** 25
GPLv3 o posterior, archivo por archivo.

NIVEL PROFUNDO: CADA FUNCIÓN

16 **web/app.js** 28
598 funciones, en el orden del archivo.

Base: ayudantes y almacén 28
Direcciones del servidor 29
API de Jellyfin 29
Audio: formas de onda y sintetizador 31
Biblioteca demo 31
Ajustes y temas 33
Descargas (Android) 33
Estado y navegación 34
Álbumes según las etiquetas 34
Búsqueda en las letras 35
Ayudantes de dibujo 35
Vistas 35
DJ Nanuq 36
Tu año y Tu semana (video) 38
Solicitar canción (SomeDL) 40
Reproductor 41
Letras 43
Cola 44
Avisos, menús y diálogos 44
Mis dispositivos 44
Fiesta (watch party) 46
Acciones globales 48
Entrar 48

17 **Resto de la página** 50
Idiomas y service worker.

web/i18n.js 50

web/sw.js 50

18 **Android** 51
Java nativo y el puente de JavaScript.

android/app/src/main/java/com/nanuq/app/Discovery.java 51
android/app/src/main/java/com/nanuq/app/Downloads.java 51
android/app/src/main/java/com/nanuq/app/LocalServer.java 52
android/app/src/main/java/com/nanuq/app/MainActivity.java 53
android/app/src/main/java/com/nanuq/app/MediaCenter.java 54
android/app/src/main/java/com/nanuq/app/NativeHttp.java 55
android/app/src/main/java/com/nanuq/app/PlaybackService.java 55
android/app/src/main/java/com/nanuq/app/Prefetch.java 56
android/app/src/main/java/com/nanuq/app/SecureStore.java 56
android/bridge/android-bridge.js 57
android/build-apk.sh 57
android/app/src/main/java/com/nanuq/app/OggCheck.java 57

19 **Escritorio** 58
Electron y su empaquetado.

linux/main.js 58
linux/preload.js 60
linux/prepare-web.js 60
linux/collect.js 60
linux/flatpak/build-flatpak.sh 60

20 **Compilación** 61
El script que compila todo.

scripts/build.sh 61

PARTE 1

Arquitectura

Cómo está armado Nanuq: una sola interfaz web, las apps que la envuelven y cómo se habla con Jellyfin, con tus otros dispositivos y con la fiesta.

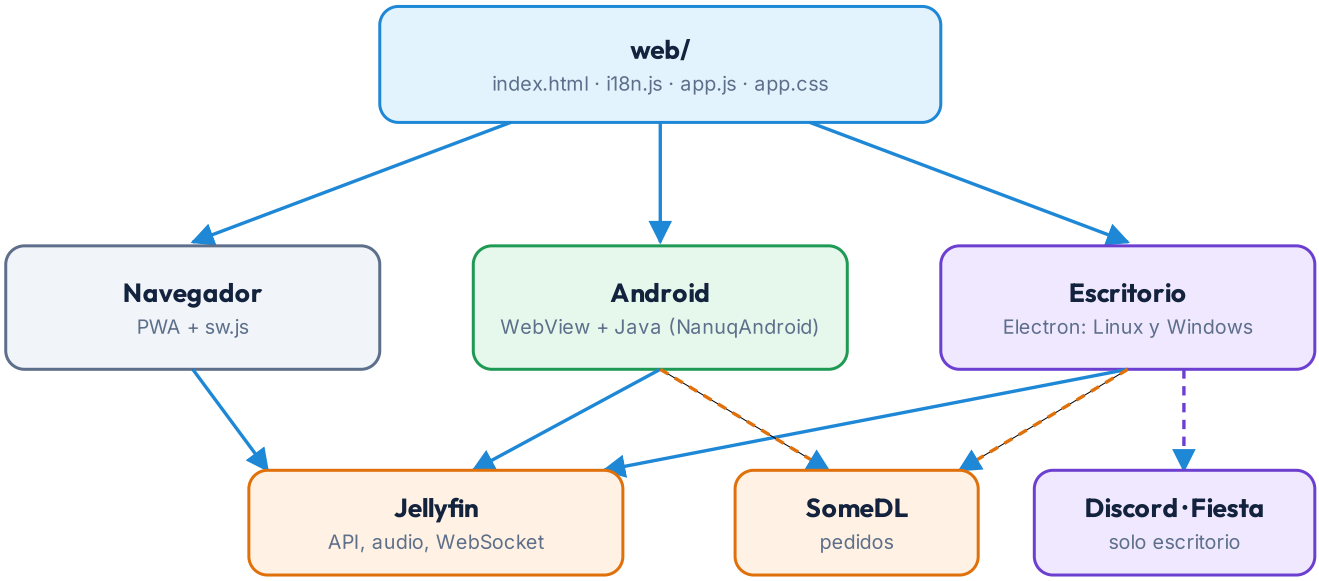


01 Panorama

Una sola interfaz, tres envolturas.



Toda la interfaz de Nanuq vive en `web/`: cuatro archivos sin compilar y sin dependencias. Las apps de Android y de escritorio no tienen interfaz propia: copian esa misma carpeta al compilar y le agregan, por un «puente», lo que una página web no puede hacer sola.



Cada plataforma sirve la misma página; las flechas punteadas son lo que solo hace el código nativo (SomeDL sin CORS, Discord, la sala de la fiesta).

Por qué así

- **Un cambio llega a todas partes**: la interfaz se toca solo en `web/`; Android y Linux la copian al compilar (`syncWeb` en Gradle, `prepare-web.js` en Electron).
- **Contexto seguro sin HTTPS**: Android sirve la página en `http://localhost` y Electron en `http://127.0.0.1:47863`. Así hay `crypto.randomUUID` y almacenamiento (el service worker solo se usa en el navegador), y a la vez se puede hablar con un Jellyfin `http://` de la red local sin bloqueos de contenido mixto.
- **Sin servicios de terceros**: Nanuq solo habla con tu Jellyfin, tu SomeDL y (si lo activas) la app de Discord de tu computadora. Las únicas peticiones externas son las fuentes de Google Fonts.

Puertos

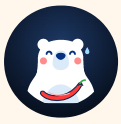
Puerto	Quién	Para qué
TCP 47863	Escritorio (127.0.0.1)	Sirve la página y, con la fiesta abierta, el audio de la sala (`/fiesta/<token>/…`). Puerto fijo porque el almacenamiento depende del origen.
TCP 47864	Escritorio (red local)	WebSocket de la fiesta (`/fiesta`) y carátulas para los invitados. Solo con la sala abierta y solo direcciones privadas.
UDP 47865	Escritorio	Responde a «Who is NanuqParty?» para que los teléfonos encuentren la sala (≤ 30 respuestas/s).
UDP 7359	Jellyfin	Búsqueda oficial de servidores («Who is JellyfinServer?»), desde Android y escritorio.
TCP aleatorio	Android (127.0.0.1)	`LocalServer` : descargas, carátulas y lo guardado por adelantado, con token en la ruta.
8096 / 8920	Jellyfin	Puertos que se asumen si escribes una IP o un nombre corto sin puerto.
5000	SomeDL	Puerto que se asume para SomeDL sin puerto.

02 El repositorio

Qué hay en cada carpeta.



Ruta	Contenido
`web/index.html`	Estructura, CSP estricta y `<defs>` SVG: íconos `#i-*` y el oso vectorizado (`#djb-*`).
`web/i18n.js`	Idiomas: `LANGS` , `LANG` , `LOC` , el diccionario `EN` (clave = texto en español) y `tx()` . Se carga antes que app.js.
`web/app.js`	Toda la lógica (6062 líneas). `const VERSION` es la fuente de verdad de la versión.
`web/app.css`	Todo el estilo, con variables por tema (18 temas).
`web/sw.js`	Service worker, solo en el navegador: guarda la app (red primero para el código), nunca música ni API. Android y escritorio no lo registran.
`android/`	WebView + Java (`com.nanuq.app`), el puente `bridge/android-bridge.js` y `build-apk.sh` .
`linux/`	Electron: `main.js` , `preload.js` , `prepare-web.js` , `collect.js` y `flatpak/` . También arma el instalador de Windows.
`scripts/build.sh`	Sube la versión y compila todo a `dist/<versión>/` .
`docs/`	Imágenes del README y estos manuales (`docs/manuals/src` : `node docs/manuals/src/build.js`).



Ojo

`dist/` , `keystore.properties` y los `*.jks` no van al repositorio. La llave de firma del APK vive fuera (`~/.android/nanuq-release.jks`): sin ella no se pueden publicar actualizaciones.

03

La página

Cómo arranca, navega y reacciona app.js.



Arranque

1. Al cargar, `i18n.js` decide el idioma y `txStatic()` traduce los textos fijos del HTML.
2. `SEC` detecta el almacén cifrado nativo y `secMigrate()` mueve allí lo que versiones viejas dejaron en claro.
3. El bloque de arranque dibuja la barra lateral y las pestañas, aplica el tema e inicia las descargas (`DL.init`).
4. `Vault.init()` reintenta unos segundos si el Keystore o el llavero aún no responden; luego `Vault.load()` lee la sesión.
5. Con sesión: `startApp(new Jellyfin(sess))` recupera la cola y la posición (`QPOS`), restaura el DJ y abre Inicio. Sin sesión: `showLogin()`.

Navegación

`S.stack` es la pila de pantallas. `go(nombre, params, opts)` empuja una pantalla (las de `TOP` vacían la pila) y llama a `render()`, que ejecuta `V[nombre](caja, params)`. Cada vista es una función asíncrona que llena su caja; si deja `_cleanup`, se llama al salir. `back()` cierra primero lo que esté encima (menú, editor de letra, reproductor) y luego retrocede. Con la fiesta abierta (`party-host` / `in-party`) solo existe la pantalla de la fiesta.

Eventos

La CSP prohíbe scripts en línea, así que no hay `onclick=`: todo botón lleva `data-act="…"` y un único manejador de clic global decide qué hacer (sección «Acciones globales»). Las imágenes que deben desaparecer si fallan llevan `data-rm`.

Idiomas

Todo texto visible se escribe en español dentro de `tx('…')`; `i18n.js` tiene la traducción con el texto en español como clave. Con valores: `tx('… {0} …', [x])`. Cambiar de idioma guarda la posición y recarga la página, así también se traducen los textos que se arman al cargar.

Secciones de app.js

Sección	Desde la línea
Base: ayudantes y almacén	1

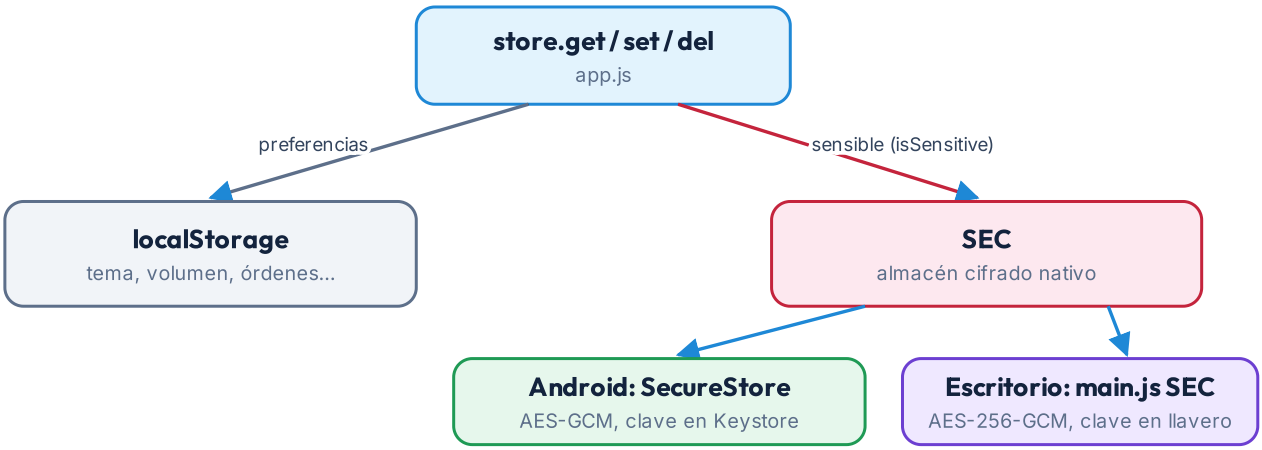
Sección	Desde la línea
Direcciones del servidor	122
API de Jellyfin	177
Audio: formas de onda y sintetizador	494
Biblioteca demo	555
Ajustes y temas	753
Descargas (Android)	804
Estado y navegación	957
Álbumes según las etiquetas	1007
Búsqueda en las letras	1112
Ayudantes de dibujo	1210
Vistas	1240
DJ Nanuq	1307
Tu año y Tu semana (video)	2263
Solicitar canción (SomeDL)	2979
Reproductor	3270
Letras	3823
Cola	4169
Avisos, menús y diálogos	4240
Mis dispositivos	4251
Fiesta (watch party)	4949
Acciones globales	5693
Entrar	5853
Arranque	6033

04 Datos y almacenamiento

Lo que dice algo de ti va cifrado.



`store` es el único acceso al almacenamiento. Las preferencias de la interfaz (tema, volumen, órdenes) van a `localStorage` con el prefijo `nanuq.`; lo que dice algo de ti va al almacén cifrado de la plataforma. Como las preferencias no van cifradas, se validan al leerlas (justo después de `Settings`): un valor alterado o de otra versión vuelve al de siempre.



En el navegador no hay almacén seguro: todo queda en localStorage (Ajustes lo dice).

Clave	Qué guarda
session	Servidor, usuario y token de Jellyfin.
queue · queuePos	Tu cola (hasta 3000) y el segundo donde ibas (queuePos no es sensible: solo un id al azar y segundos).
libIdx · lyrIdx	Índices de tu biblioteca (álbumes por etiquetas) y de tus letras.
dj · djHist	Sesión del DJ en curso e historial de lo que puso.
plays · mp3ids	Reproducciones por canción en este dispositivo y canciones que van en MP3.
somedl · somedlReqs · somedlLast	Servidor SomeDL y tus pedidos.
servers · lastServer · device · party	Servidores recientes, id del dispositivo y la fiesta en curso (teléfono).
lrc.<id>	Letras guardadas solo en este dispositivo.

Estados del almacén

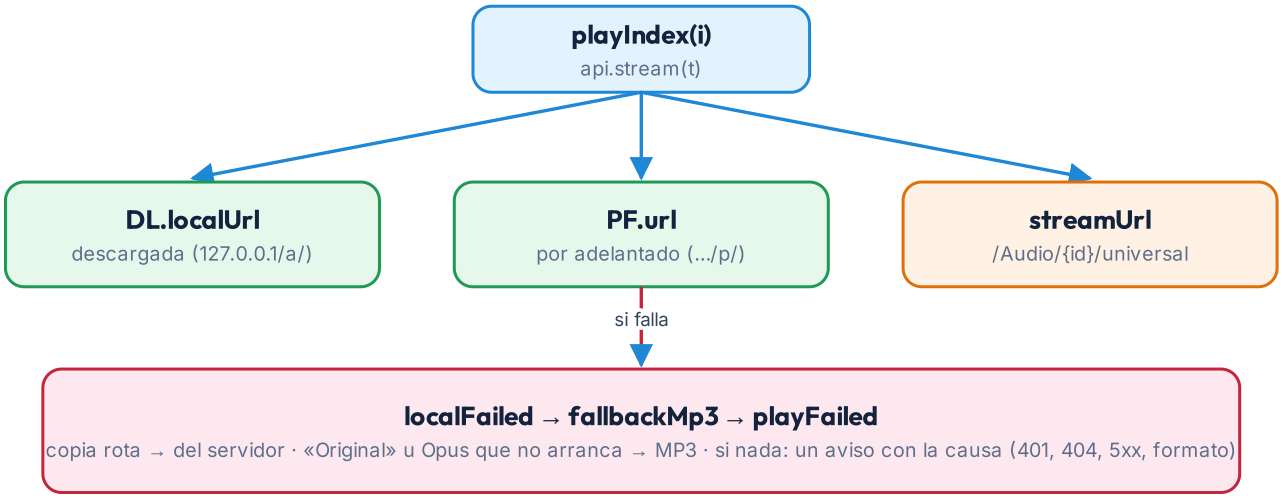
`SEC.kind` puede ser el nombre del almacén (funciona), `locked` (hay datos cifrados que ahora no abren: se reintenta), `lost` (la clave ya no existe: se ofrece «Empezar de cero», que aparta los datos sin borrarlos) o `null` (no hay almacén). Mientras no esté abierto, lo nuevo vive solo en memoria (`SEC.mem`): **nunca** se escribe en claro.

05 Reproducción

De dónde sale cada canción y qué pasa cuando algo falla.



`P` guarda la cola, el índice y dos `<audio>` (`P.els`; `P.audio` es siempre el de la canción actual). `playIndex(i)` elige la fuente con `api.stream()`: primero la copia descargada, luego la guardada por adelantado y, si no hay, el streaming de Jellyfin.



En Android, las dos primeras fuentes las sirve el teléfono mismo; en Android y escritorio la página nunca baja el audio completo a memoria.

Detalles que importan

- **Saltar en conversiones en vivo**: si Jellyfin transcodifica, el archivo no admite saltos; `seekTo` pide el audio otra vez con `StartTimeTicks` y recuerda el desplazamiento (`P.off`). Al arrastrar, la petición sale al soltar.
- **Atascos**: `armStallWatch` avisa si no arranca en 10 s; `stallCheck` reanuda desde el mismo segundo si la red se cayó a media canción; `netBack` lo hace al volver la red.
- **Formatos**: `streamUrl` deja sonar tal cual FLAC, Opus, AAC, MP3, WAV y OGG (en «Original» o bajo el bitrate); si hay que convertir, pide Opus en Ogg (`Settings.codec`, si el reproductor lo abre: `OPUS_OK`) o MP3. Lo que falló una vez va en MP3 (`MP3.ids`). Las descargas usan el mismo formato.
- **Desvanecer**: `pauseSoft` / `playSoft` con `ramp()`, lineal en decibelios (−50 dB = silencio), de 0.5 a 5 s (`Settings.fadeMs`). Mientras baja ya cuenta como pausa (`FADE.out`) y ▶ lo revierte. El volumen de vuelta es siempre el tuyo (`userVol`). Al arrastrar la barra, rampas de 30–40 ms solo contra chasquidos (`P.scrub`).
- **Transición del DJ**: `xfCheck` en `timeupdate`; a 3–5 s del final (`Settings.xfade`), `xfStart` intercambia los dos `<audio>` y cruza con curvas de potencia constante (seno/coseno). Los eventos del que se va se ignoran (`onAudio`); `xfKill` lo apaga al pausar o cambiar de canción.

- **Pausa ajena**: si otra app tomó la salida de audio (`P.extPause`), al reanudar se recarga la canción para no quedarse mudo.
- **Controles del sistema**: `navigator.mediaSession`; en Android lo reemplaza el puente nativo. En escritorio un audio en silencio (`rcKeep` / `sessionHold`) mantiene los controles mientras carga o controlas otro dispositivo.
- **Favoritas**: Jellyfin 12 sobrescribe la favorita de la canción que suena con cada aviso de reproducción; `FAVHOLD` la vuelve a guardar después de cada aviso.

06

Biblioteca y Jellyfin

La API que usa Nanuq y lo que arma por su cuenta.



Ruta de Jellyfin	Para qué
/System/Info/Public · /Users/AuthenticateByName	Reconocer el servidor y entrar.
/Items · /Artists/AlbumArtists · /MusicGenres · /Playlists	Listas de la biblioteca (con varias rutas de respaldo según la versión).
/Audio/{id}/universal	El audio (streaming, descargas, por adelantado, fiesta).
/Audio/{id}/Lyrics · .../RemoteSearch/Lyrics	Leer, subir y buscar letras (10.9+).
/UserFavoriteItems/{id}	Favoritas (y la ruta anterior a 10.9 como respaldo).
/Items/{id}/InstantMix	Radio de una canción.
/Sessions · /Sessions/{id}/Playing · /socket	Mis dispositivos: sesiones, órdenes y avisos en vivo.
/Sessions/Playing(/Progress|/Stopped)	Avisos de reproducción (lo que ve el servidor).
/DisplayPreferences/nanuq-dj	Estado del DJ compartido entre tus dispositivos.
/user_usage_stats/...	Plugin Playback Reporting para Tu semana / Tu año (si está).

Álbumes por etiquetas (LIB)

Jellyfin arma los álbumes por carpeta; Nanuq los arma con las etiquetas «Álbum» + «Artista del álbum» de cada canción (`albumKeyOf`), junta «(Disc 2)» con su disco y quita duplicados. El índice de todas tus canciones se pide de a 1000 (tres páginas a la vez), se guarda cifrado y se renueva cada 10 minutos. Los ids de estos álbumes empiezan con `m` (`albumGid`).

Búsqueda en las letras (LYRX)

Jellyfin no busca dentro de las letras: Nanuq indexa las canciones con `HasLyrics` (de a 3, hasta 8000) y las letras locales, normalizadas sin acentos ni signos (`lyrNorm`). Se busca con 4 caracteres o más.

07

DJ Nanuq

Rondas con reglas, sin IA.



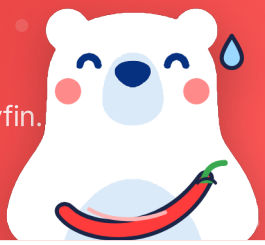
`DJ.next()` ordena los tipos de ronda al azar según su peso, evita repetir los recientes y prueba hasta encontrar una que **explore** (canciones que no sonaron en las últimas semanas). Cada canción lleva su ronda en el id de la cola: `dj—<sesión>-<ronda>-<azar>`.

Ronda	Peso	Qué pone
genre	6	Un género de tu biblioteca (≥ 3 canciones en una muestra de 400), sin repetir los recientes.
decade	3.5	Una década real de tu biblioteca; las pasadas pesan ×1.5.
discover	3	Nunca escuchadas, primero en tus géneros favoritos.
forgotten	3	Más de 45 días sin sonar.
mood	1	Según la hora (mañana tranquila, tarde con energía…).
fresh · fav · recent · top	.8 · .6 · .15 · .15	Recién llegadas, favoritas, lo de estos días y lo más escuchado (con enfriamiento).

`DJ.rank()` decide qué tan «gastada» está una canción: lo de los últimos 3 días o de sesiones recientes del DJ casi no entra; lo olvidado va primero. Al pasar la música a otro dispositivo, `DJSHARE` deja el estado en tu Jellyfin y el otro lo adopta (`DJ.adopt`); lo que se lee se limpia con `djClean`. Mientras se arma la primera ronda, `DJL` muestra a Nanuq en su tornamesa con frases (`DJ_LOAD_SAYS`) hasta que suena.

08 Mis dispositivos

Un Nanuq suena y los demás lo controlan, a través de Jellyfin.





No se abre ningún puerto en los dispositivos: todo pasa por tu Jellyfin, con tu sesión.

- **Quién aparece**: sesiones con cliente «Nanuq», de tu mismo usuario, activas hace menos de 15 minutos según el reloj del *servidor* (los relojes desfasados no estorban).
- **Misma red**: misma /24 en IPv4 o /64 en IPv6 (o ambas privadas), con la dirección que ve el servidor.
- **Órdenes aceptadas**: `Play` y `GeneralCommand` solo si las manda tu usuario (dato que pone Jellyfin); `Playstate` lo filtra Jellyfin por permisos.
- **Controlando**, tu cola local se aparta (`RC.localQ`) y vuelve al desconectarte; la posición se estima desde el último reporte (`RC.anchor`).
- En Android, con la pantalla bloqueada, la app nativa llama a `__nanuqTick` cada 20 s para mantener vivo el WebSocket.

09

La fiesta

Teléfonos que mandan canciones a una computadora por la red local.





UDP :47865 «Who is NanuqParty?» → nombre y puerto de la sala

El teléfono baja la canción de SU Jellyfin y se la pasa a la computadora en trozos de 256 KB, sin tenerla entera en memoria.

Mensaje	Sentido	Qué es
hello {code|key, name, theme}	teléfono → PC	Entrar con el código de 6 dígitos o volver con su llave.
welcome {you, key, room, chunk}	PC → teléfono	Entró: su llave para volver sin código (2 h).
add · [id + bytes] · end · remove	teléfono → PC	Mandar una canción (datos, letra, carátula) y sus bytes.
addOk · addErr · ready	PC → teléfono	Respuesta al pedido y aviso de que ya está lista.
state · tick	PC → teléfonos	Lista, invitados y lo que suena; segundo y línea del karaoke.
bye {why} · leave · ping	ambos	Salir, ser sacado o avisar que sigue ahí.

Topes y defensas

`PT_MAX` en `main.js`: `{ guests: 12, conns: 24, file: 80e6, disk: 800e6, list: 60, perGuest: 10, frame: 1 << 20, text: 450e3, lyrics: 200e3, cover: 150e3 }`. Solo direcciones privadas (`ptLocal`), ≤ 4 conexiones sin presentarse por dirección, 5 códigos malos por minuto por dirección y, con 30 fallos en total, código nuevo y un minuto cerrado. El JSON se lee con `jparse` y los textos con `ptStr`; un mensaje que haga fallar algo solo corta esa conexión.

Canciones en disco, cifradas

Las canciones van a una carpeta en disco (`ptBase()`: `~/.cache/nanuq/nanuq-fiesta-*` en Linux; en el Flatpak, dentro de `~/.var/app/com.nanuq.app/cache`), no a la temporal, que allí es memoria RAM. Se

rechaza una canción si quedan menos de 1 GB libres. Cada canción se guarda **cifrada y autenticada** con AES-256-GCM en bloques de 64 KB (`PT_CH`): la clave es al azar y solo vive en la memoria de la sala (`PT.fkey`), y el nonce es de 8 bytes al azar por canción más el número de bloque (`ptNonce`). `ptSeal` cifra al llegar; `ptServe` comprueba el tamaño exacto (`ptEncSize`) y `ptOpenRange` descifra solo los bloques del rango pedido. Si otra app cambia un byte, corta el archivo o cambia bloques de lugar, el bloque no se entrega y la canción sale de la lista.

10 La app de Android

WebView + Java nativo.

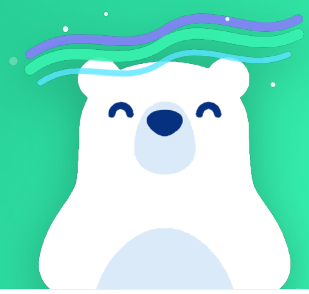


Clase	Responsabilidad
MainActivity	WebView (assets en `http://localhost`), el puente `NanuqAndroid`, latido de 20 s, recrear el WebView si muere, enlaces externos solo http(s).
MediaCenter · PlaybackService	MediaSession y notificación; servicio en primer plano con bloqueos de CPU y Wi-Fi mientras suena; se libera tras 10 min en pausa.
Downloads	Descargas de una en una, solo Wi-Fi por omisión, límite con reemplazo de automáticas; índice cifrado; Opus solo si sus páginas Ogg están completas (`OggCheck`).
Prefetch	Lo que sigue en la cola, en la caché (≤ 400 MB por archivo, 1 GB total, 300 MB libres).
LocalServer	HTTP en 127.0.0.1 con token: `a/` audio, `c/` carátula, `p/` por adelantado; Range; 10 s por petición, ≤ 100 encabezados, 32 hilos.
SecureStore	Almacén AES-GCM con clave en Android Keystore; estados locked / lost.
NativeHttp · Discovery	HTTP sin CORS solo para las rutas de SomeDL; búsqueda UDP de Jellyfin y de fiestas.

El puente `android-bridge.js` se inyecta en `index.html` al compilar y reemplaza `navigator.mediaSession` por uno que habla con `MediaCenter`. Solo se descargan direcciones con forma de Jellyfin (`Downloads.okUrl`) y las redirecciones solo dentro del mismo servidor (`openSameHost`).

11 La app de escritorio

Electron para Linux y Windows.



Canal IPC	Qué hace
nanuq:discover	Busca servidores Jellyfin por UDP.
nanuq:http	HTTP sin CORS, solo rutas de SomeDL, sin redirecciones, ≤ 4 MB.
nanuq:sec (sync) · nanuq:secreset	Almacén cifrado: clave AES-256 aleatoria guardada con `safeStorage` (llavero / DPAPI).
nanuq:discord	Rich Presence por el socket `discord-ipc-N` (solo sockets tuyos), con reintento mínimo y estado para Ajustes.
nanuq:party · nanuq:party-ev	Abrir/cerrar la sala y operarla (jump, order, played, now, tick, lyrics).

- Todo canal IPC comprueba `fromApp(e)`: solo la página de nuestro origen puede pedirlo.
- Ventana con `contextIsolation`, `sandbox` y sin Node; permisos solo notificaciones, pantalla completa y portapapeles.
- Fusibles de Electron: sin `runAsNode` ni `NODE_OPTIONS`, integridad del asar y solo cargar desde el asar.
- **Integridad del código:** sin caché HTTP en disco (`disable-http-cache`; la carpeta `Cache` vieja se borra al arrancar) y, antes de abrir la página, se borran el service worker, Cache Storage y la caché de código de V8 (también al recargar con F5). `window.json` solo acepta números razonables.
- Servidor local: solo GET/HEAD, `Host` 127.0.0.1/localhost (contra DNS rebinding), rutas ≤ 512 y sin bytes nulos.

12 Seguridad

Lo que llega de afuera nunca se cree.



- **CSP** sin scripts en línea, sin `eval`, sin iframes ni `<base>`; solo atributos `style`.
- **JSON de afuera** con `jparse` (sin `toString` / `valueOf` / `__proto__`) y textos con `str()`: nunca `String(x)` sobre datos ajenos.
- **HTML**: todo dato pasa por `esc()`; los ids por `sid()` (solo letras, números y guiones) antes de ir a una ruta.
- **Letras** limpias de control y etiquetas (`cleanLyricsText`), ≤ 200 000 caracteres.
- **Puentes nativos** con lista blanca de rutas y topes de tamaño; nada de seguir redirecciones a otro servidor.
- **Datos personales** cifrados en disco; respaldo de Android desactivado (`allowBackup=false` y reglas de extracción).
- **Integridad en disco**: nada de lo guardado se usa sin comprobar. Lo cifrado (datos personales, índice de descargas, canciones de la fiesta) va con AES-GCM, que detecta cualquier cambio; las preferencias sin cifrar se validan al leerlas; en Android todo vive en el almacenamiento privado de la app y una descarga Opus solo se guarda si todas sus páginas Ogg llegan completas con su CRC (`OggCheck.valid`; las conversiones no traen tamaño); el tipo de audio del servidor solo se acepta como `audio/…` (`OggCheck.mime`).



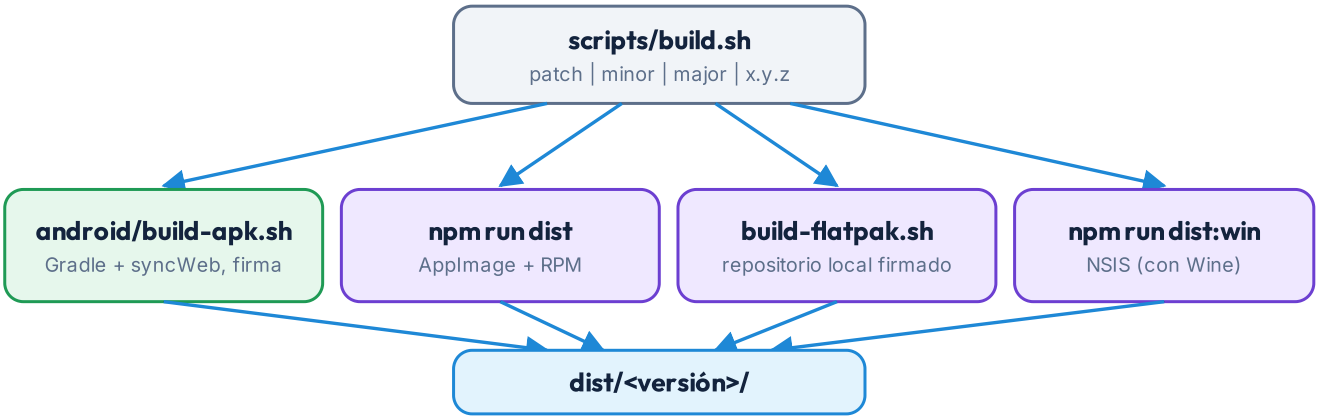
Consejo de Nanuq

El detalle completo (qué se protege y los límites conocidos) está en `SECURITY.md`.

13 Compilar y publicar

Una orden para todas las plataformas.





La versión vive en `web/app.js` (VERSION) y en `web/sw.js` (CACHE); el APK, el package.json y el Flatpak la toman de ahí.

- **Versionado**: funciones nuevas suben el número menor; arreglos, el último. El `versionCode` del APK es M·10000 + m·100 + p.
- **Android** necesita JDK 21, el SDK y `android/keystore.properties`. Firmado con la misma llave siempre.
- **Linux**: el RPM usa el `fpm` del sistema (`USE_SYSTEM_FPM=true`). El Flatpak se arma con `linux-unpacked`.
- **Comprobar**: `node --check web/app.js` y, para interfaz, Chromium en modo demo.

14 Convenciones y trampas

Lo que conviene saber antes de cambiar algo.



Regla	Por qué
Cambios de interfaz solo en `web/`	Android y Linux la copian al compilar.
Todo texto visible en `tx('…')` + su traducción en `EN`	Si no, sale en español con el idioma en inglés. Una frase completa por `tx`, sin pegar trozos.
Nada de `onclick=`, `<script>` ni `<style>` desde JS	La CSP los bloquea sin avisar.
Ids únicos en toda la app	Un id repetido escondió el cuadro de Buscar (1.30.2).
`view-transition-name` únicos	Si se repiten, la transición se cancela sin error visible.
Animaciones sin fin solo en ráfagas (`nqBurst`)	A 120 Hz cada una cuesta ≈ 1 núcleo del teléfono.
Respetar `prefers-reduced-motion`	Accesibilidad.
Tras escribir en Jellyfin, mostrar lo local y confirmar después	La lectura inmediata puede devolver lo viejo (letras, playlists, favoritas).
`.ig-crowd` sin `z-index`	Crearía su propia capa y todos quedarían detrás del DJ.

15 Licencia

GPLv3 o posterior, archivo por archivo.



Nanuq es software libre bajo la **GNU General Public License, versión 3 o (a elección de quien lo reciba) cualquier versión posterior**: `GPL-3.0-or-later`. El texto completo está en `LICENSE`, en la raíz del repositorio.

Qué significa para quien lo modifica

- Puedes usar, estudiar, copiar y modificar el código para lo que quieras.
- Si **distribuyes** Nanuq o una versión modificada (un APK, un paquete, la PWA en un servidor que otros descargan), debe ir bajo la misma licencia y con su código fuente (o una oferta para obtenerlo), con los avisos de copyright y de licencia intactos.
- Una versión modificada debe decir que lo es. Para Android, usa otro `applicationId` y tu propia llave de firma: el APK oficial solo lo firma su autor (ver «Compilar y publicar»).
- Se ofrece **sin ninguna garantía**.

Cómo está marcado el código

Cada archivo fuente empieza con dos líneas en el formato de **SPDX** (el que usan las herramientas de licencias y la especificación REUSE):

```
SPDX-FileCopyrightText: 2026 Fleyrer y colaboradores de Nanuq
SPDX-License-Identifier: GPL-3.0-or-later
```

Van como comentario del lenguaje de cada archivo (`//` , `/* */` , `<!-- -->` o `#`), después del `#!` en los scripts y del `<!DOCTYPE>` en el HTML. Al crear un archivo nuevo, agrégale las mismas dos líneas.

Dónde más aparece

Lugar	Cómo
`LICENSE`	Texto oficial de la GPLv3 (GitHub lo detecta y lo muestra en el repositorio).
README.md · README.en.md	Línea arriba, sección «Licencia» y enlace en el menú.
Ajustes de la app	Fila «Nanuq x.y.z»: la licencia y un enlace a `LICENSE` (se abre en el navegador).
linux/package.json	`"license": "GPL-3.0-or-later"`
Flatpak (metainfo)	`<project_license>GPL-3.0-or-later</project_license>`
Estos manuales	Portada y este capítulo.

Lo que no es de Nanuq

Componente	Licencia	Dónde
Electron	MIT	Dentro de las apps de escritorio (Linux y Windows).
Inter · Outfit	SIL Open Font License	Fuentes, cargadas de Google Fonts (no van en el repositorio).
Gradle wrapper	Apache 2.0	`android/gradlew`
Jellyfin	GPL-2.0	El servidor: un proyecto aparte con el que Nanuq habla por su API. Nanuq no está afiliado a él.

PARTE 2

Nivel profundo: cada función

Las 860 funciones y métodos del código, archivo por archivo, con la línea donde empiezan y qué hacen.



16 web/app.js

598 funciones, en el orden del archivo.



web/app.js 6062 líneas

Base: ayudantes y almacén desde la línea 1 · 25

Función	Línea	Qué hace
$	5	Atajo de document.querySelector (opcionalmente dentro de otro elemento).
$$	6	Atajo de querySelectorAll que devuelve un arreglo.
jparse	10	Lee JSON de afuera sin toString/valueOf/__proto__.
str	11	Convierte cualquier valor a texto sin lanzar nunca un error.
esc	12	Escapa un texto para meterlo en HTML (& < > " ').
sid	16	Limpia un id del servidor: solo letras, números y guiones.
num	17	Convierte a número; si no es finito, 0.
numOrNull	18	Convierte a número o devuelve null si está vacío o no es válido.
ic	30	HTML de un ícono SVG de las <defs> (#i-nombre).
fmt	31	Formatea segundos como m:ss o h:mm:ss.
isSensitive	38	¿Esta clave guardada contiene datos personales (va al almacén cifrado)?
SEC.ok	46	¿El almacén cifrado nativo está abierto y usable?
SEC.shut	47	¿El almacén está bloqueado o perdido (no se escribe nada en claro)?
store.get	55	Lee un valor guardado (cifrado si es sensible, si no localStorage) y migra copias viejas en claro.
store.set	67	Guarda un valor; lo sensible va al almacén cifrado o, si está cerrado, solo a memoria.
store.del	78	Borra un valor guardado en el almacén y en localStorage.
secMigrate	81	Pasa al almacén cifrado lo que versiones anteriores dejaron en claro.
Vault.kind	95	Nombre del almacén cifrado en uso (o null).
Vault.init	98	Arranque en frío: reintenta abrir el almacén unos segundos antes de rendirse.
Vault.save	101	Guarda la sesión de Jellyfin.

Función	Línea	Qué hace
`Vault.load`	102	Lee la sesión guardada (o marca que el almacén está bloqueado).
`Vault.reset`	109	«Empezar de cero»: aparta los datos ilegibles y crea un almacén nuevo.
`uid`	117	Id aleatorio (UUID sin guiones).
`deviceId`	118	Id persistente de este dispositivo para Jellyfin.
`deviceName`	119	Nombre del sistema (Android, Linux, Windows…) que se anuncia a Jellyfin.

Direcciones del servidor desde la línea 122 · 1

Función	Línea	Qué hace
`parseServer`	124	Interpreta lo que escribiste como servidor (IPv4, IPv6, nombre, puerto y ruta) y arma la URL base.

API de Jellyfin desde la línea 177 · 58

Función	Línea	Qué hace
`Jellyfin.constructor`	180	Crea el cliente de Jellyfin a partir de la sesión guardada.
`Jellyfin.authHeader`	181	Arma la cabecera de autorización MediaBrowser con dispositivo, versión y token.
`Jellyfin.req`	185	Petición a la API con tiempo límite, sin caché; detecta 401 y lee el JSON con jparse.
`Jellyfin.alt`	203	Prueba varias rutas de la API hasta que una no dé 404.
`Jellyfin.probe`	204	Comprueba que una dirección sea un Jellyfin (/System/Info/Public).
`Jellyfin.login`	212	Inicia sesión con usuario y contraseña y guarda el token.
`Jellyfin.cfg`	216	Datos de la sesión para guardar.
`Jellyfin.map`	217	Convierte un elemento de Jellyfin al formato de Nanuq, limpiando ids, números y textos.
`Jellyfin.items`	236	Consulta genérica de /Items con los campos que usa la app.
`Jellyfin.albums`	240	Álbumes de Jellyfin paginados y ordenados.
`Jellyfin.item`	244	Un elemento por id (con ruta de respaldo para servidores viejos).
`Jellyfin.album`	245	Un álbum de Jellyfin y sus pistas.
`Jellyfin.artists`	249	Artistas del álbum, paginados.
`Jellyfin.artistFilters`	254	Filtros posibles para encontrar las cosas de un artista (por varios ids o por nombre).
`Jellyfin.firstNonEmpty`	260	Prueba filtros en orden y devuelve el primer resultado que no esté vacío.

Función	Línea	Qué hace
`Jellyfin.artist`	264	Un artista y sus álbumes (armados con sus canciones si hace falta).
`Jellyfin.artistTracks`	279	Todas las canciones de un artista.
`Jellyfin.genres`	287	Géneros de música probando varias rutas; si no, los saca de los álbumes.
`Jellyfin.genreParam`	306	Parámetros de consulta para uno o varios géneros (por id o por nombre).
`Jellyfin.genre`	311	Álbumes de un género.
`Jellyfin.genreTracks`	318	Canciones de un género (de las etiquetas de las canciones o de sus álbumes).
`Jellyfin.playlists`	327	Tus playlists de audio.
`Jellyfin.playlist`	328	Una playlist y sus canciones.
`Jellyfin.views`	332	Bibliotecas de música del usuario (raíz de Carpetas).
`Jellyfin.folder`	336	Contenido de una carpeta.
`Jellyfin.search`	337	Búsqueda de artistas, álbumes y canciones.
`Jellyfin.img`	344	URL de la carátula (local si está descargada en el teléfono).
`Jellyfin.streamUrl`	351	URL de streaming universal: formatos que suenan tal cual y conversión a Opus o MP3.
`Jellyfin.stream`	357	URL para reproducir: descarga, caché por adelantado o streaming.
`Jellyfin.dlUrl`	360	URL para descargar una canción al teléfono (Opus o MP3, o el archivo tal cual).
`Jellyfin.peaks`	367	Baja una copia ligera y calcula la forma de onda real.
`Jellyfin.tracks`	372	Canciones de la biblioteca paginadas y ordenadas.
`Jellyfin.lyrics`	376	Letra de una canción (Jellyfin 10.9+), normalizada.
`Jellyfin.mix`	381	Canciones para una mezcla (orden, filtros, géneros, años).
`Jellyfin.uploadLyrics`	388	Sube una letra (.lrc o .txt) a Jellyfin.
`Jellyfin.searchLyrics`	395	Busca letras en los proveedores de Jellyfin (LrcLib).
`Jellyfin.pickLyrics`	399	Elige una de las letras encontradas para guardarla.
`Jellyfin.setFavorite`	401	Marca o quita una favorita.
`Jellyfin.inFavorites`	407	¿Está en la sección de favoritas de Jellyfin?
`Jellyfin.isFavorite`	408	¿Esta canción es favorita según sus datos de usuario?
`Jellyfin.favPlaylist`	415	Encuentra la playlist del plugin «Favorited Songs Playlist».
`Jellyfin.favorites`	422	Tus favoritas (filtro IsFavorite), paginadas y ordenadas.
`Jellyfin.setFavoriteLegacy`	440	Ruta antigua para guardar favoritas (segundo intento).
`Jellyfin.instantMix`	442	Mezcla instantánea de Jellyfin a partir de una canción (radio).

Función	Línea	Qué hace
`Jellyfin.addToPlaylist`	447	Agrega canciones a una playlist.
`Jellyfin.refreshLibrary`	449	Pide a Jellyfin que vuelva a escanear las bibliotecas (administradores).
`Jellyfin.savePlaylist`	450	Crea una playlist nueva con canciones.
`Jellyfin.report`	451	Avisa a Jellyfin que empezó, avanza o terminó una canción (y reafirma favoritas).
`Jellyfin.logout`	457	Cierra la sesión en el servidor (revoca el token).
`Jellyfin.wsUrl`	460	URL del WebSocket de Jellyfin.
`Jellyfin.postCapabilities`	461	Anuncia que este Nanuq acepta órdenes de control remoto.
`Jellyfin.sessions`	463	Sesiones activas (para Mis dispositivos).
`Jellyfin.sessionCmd`	464	Manda una orden de reproducción a otra sesión.
`Jellyfin.sessionPlay`	465	Manda una cola para reproducir en otra sesión.
`Jellyfin.sessionGeneral`	466	Manda una orden general (repetir, mezclar) a otra sesión.
`Jellyfin.itemsByIds`	467	Pide varios elementos por id, de 100 en 100.
`netError`	473	Mensaje claro cuando no hay conexión con el servidor.
`normLyrics`	482	Normaliza una letra de Jellyfin (sincronizada o en texto).

Audio: formas de onda y sintetizador

desde la línea 494 · 6

Función	Línea	Qué hace
`actx`	497	AudioContext compartido (creado al primer uso).
`decode`	498	Decodifica audio a un AudioBuffer.
`peaksFromBuffer`	499	Calcula los picos de la forma de onda de un AudioBuffer.
`rng`	506	Generador pseudoaleatorio con semilla.
`wav`	507	Convierte un AudioBuffer en un archivo WAV.
`synth`	514	Sintetiza una canción de prueba para el modo demo.

Biblioteca demo

desde la línea 555 · 43

Función	Línea	Qué hace
`Demo.constructor`	558	Crea la biblioteca de demostración (artistas, álbumes, canciones y playlists).
`Demo.cfg`	593	Sesión del modo demo.
`Demo.wait`	594	Simula la espera de la red.
`Demo.albums`	595	Versión demo de Jellyfin.albums.
`Demo.album`	600	Versión demo de Jellyfin.album.
`Demo.artists`	601	Versión demo de Jellyfin.artists.
`Demo.artistBy`	603	Busca un artista demo por id o por nombre.

Función	Línea	Qué hace
`Demo.artist`	604	Versión demo de Jellyfin.artist.
`Demo.artistTracks`	605	Versión demo de Jellyfin.artistTracks.
`Demo.genres`	606	Versión demo de Jellyfin.genres.
`Demo.genreTracks`	607	Versión demo de Jellyfin.genreTracks.
`Demo.genre`	608	Versión demo de Jellyfin.genre.
`Demo.playlists`	609	Versión demo de Jellyfin.playlists.
`Demo.playlist`	610	Versión demo de Jellyfin.playlist.
`Demo.views`	611	Versión demo de Jellyfin.views.
`Demo.folder`	612	Versión demo de Jellyfin.folder.
`Demo.search`	617	Versión demo de Jellyfin.search.
`Demo.img`	621	Dibuja una carátula SVG generada para el modo demo.
`Demo.buf`	635	Audio sintetizado de una canción demo (en caché).
`Demo.stream`	636	URL blob del WAV de una canción demo.
`Demo.peaks`	637	Forma de onda de una canción demo.
`Demo.tracks`	638	Versión demo de Jellyfin.tracks.
`Demo.mix`	644	Versión demo de Jellyfin.mix.
`Demo.lyrics`	654	Letra sincronizada inventada para una canción demo.
`Demo.setFavorite`	666	Versión demo de Jellyfin.setFavorite.
`Demo.isFavorite`	667	Versión demo de Jellyfin.isFavorite.
`Demo.inFavorites`	668	Versión demo de Jellyfin.inFavorites.
`Demo.favorites`	669	Versión demo de Jellyfin.favorites.
`Demo.instantMix`	670	Versión demo de Jellyfin.instantMix.
`Demo.addToPlaylist`	671	Versión demo de Jellyfin.addToPlaylist.
`Demo.savePlaylist`	672	Versión demo de Jellyfin.savePlaylist.
`Demo.report`	673	No hace nada (el demo no reporta reproducciones).
`Demo.logout`	674	No hace nada (sin servidor).
`logoInner`	679	HTML del oso del logo con el accesorio del tema.
`bearHTML`	695	HTML del Nanuq animable (logo + cara expresiva).
`bearState`	697	Estado actual del oso (bailando, contento, triste) para los osos nuevos.
`nqBurst`	703	Activa una ráfaga de animaciones por unos segundos (ahorro de batería).
`nqBurstEnd`	711	Termina la ráfaga: las animaciones se pausan y los osos quedan contentos.
`bearDance`	716	Hace bailar a todos los Nanuqs un rato.

Función	Línea	Qué hace
bearInit	721	Pone el Nanuq en los logos de la barra lateral.
bearMood	726	Cambia el ánimo de los osos al sonar o pausar la música.
showSplash	737	Pantalla de bienvenida con el Nanuq del tema.
refreshBear	748	Cambia el accesorio de los osos al cambiar de tema.

Ajustes y temas desde la línea 753 · 2

Función	Línea	Qué hace
OPUS_OK	756	¿El reproductor abre Opus? Decide si se ofrece y si va por omisión.
Settings.set	785	Cambia un ajuste y lo guarda.

Descargas (Android) desde la línea 804 · 29

Función	Línea	Qué hace
DL.init	810	Carga las descargas del teléfono desde la app nativa.
DL.indexCovers	818	Índice de carátulas descargadas por id de imagen.
DL.coverUrl	822	URL local de una carátula descargada.
DL.backfillCovers	824	Completa carátulas y datos de descargas viejas o recuperadas.
DL.config	831	Manda a Android el máximo de canciones y si solo con Wi-Fi.
DL.usable	832	¿Hay descargas en este dispositivo (Android con Jellyfin)?
DL.get	833	La descarga de una canción, si existe.
DL.localUrl	834	URL local de una canción descargada y lista.
DL.isLocal	835	¿Esta URL es del servidor local del teléfono?
DL.add	837	Pide descargar una canción (manual o automática).
DL.remove	842	Quita una canción de las descargas.
DL.removeAll	843	Borra todas las descargas (o solo las automáticas).
DL.list	844	Lista de descargas.
DL.bytes	845	Espacio total que ocupan las descargas.
DL.net	846	Tipo de red actual según Android (wifi, datos, ninguna).
__nanuqDl	848	Recibe de Android los cambios de una descarga y actualiza la vista.
Plays.count	863	Veces que sonó una canción en este teléfono.
Plays.add	864	Cuenta una reproducción y guarda sola la canción a las 3.
PF.usable	881	¿Se puede guardar por adelantado (Android con Jellyfin)?
PF.isUrl	882	¿Esta URL es de lo guardado por adelantado?

Función	Línea	Qué hace
`PF.url`	883	URL local de una canción ya guardada por adelantado (o null).
`PF.soon`	884	Programa una actualización de la ventana por adelantado.
`PF.sync`	885	Calcula qué canciones de la cola guardar (N minutos) y se lo pide a Android.
`fmtMB`	900	Formatea bytes como MB o GB según el idioma.
`dlStatus`	901	Texto del estado de una descarga.
`dlStatusShort`	912	Estado corto de una descarga para el botón del reproductor.
`dlSummary`	918	Resumen de las descargas (cantidad, espacio, pendientes).
`themeSummary`	942	HTML del tema elegido en Ajustes.
`applyTheme`	946	Aplica un tema (colores, barra del sistema, osos).

Estado y navegación desde la línea 957 · 8

Función	Línea	Qué hace
`regList`	963	Registra una lista de canciones para que los botones la encuentren.
`cur`	972	Pantalla actual de la pila de navegación.
`go`	973	Navega a una pantalla (raíz o encima de la actual).
`back`	982	Vuelve atrás: cierra diálogos, el reproductor o la pantalla actual.
`cacheDrop`	992	Olvida las vistas en caché que empiezan con un prefijo.
`refreshIf`	994	Redibuja la pantalla actual si cumple una condición.
`playlistsChanged`	997	Olvida y vuelve a pedir una playlist que cambió.
`cached`	1004	Caché de vistas con tope (las más viejas salen primero).

Álbumes según las etiquetas desde la línea 1007 · 15

Función	Línea	Qué hace
`albumNorm`	1013	Normaliza un nombre para comparar álbumes.
`albumKeyOf`	1015	Clave del álbum de una canción según sus etiquetas (artista + álbum + disco).
`albumGid`	1022	Id estable de un álbum armado por etiquetas.
`LIB.who`	1025	Identifica servidor y usuario del índice.
`LIB.index`	1027	Índice de álbumes por etiquetas (desde caché cifrada o del servidor).
`LIB.peek`	1040	El índice si ya está listo, sin esperar.
`LIB.reset`	1041	Marca el índice como viejo para renovarlo.
`LIB.refresh`	1042	Vuelve a armar el índice con todas las canciones.

Función	Línea	Qué hace
`LIB.fetchAll`	1048	Baja todas las canciones de la biblioteca (3 páginas a la vez).
`LIB.pack`	1058	Versión compacta de una canción para guardar el índice.
`LIB.unpack`	1059	Lee una canción compacta del índice guardado, limpiándola.
`LIB.save`	1064	Guarda el índice cifrado (si cabe).
`LIB.build`	1069	Agrupa las canciones en álbumes, sin repetidas.
`LIB.albumsOf`	1094	Álbumes (por etiquetas) de un grupo de canciones.
`LIB.sorted`	1099	Lista de álbumes ordenada.

Búsqueda en las letras desde la línea 1112 · 10

Función	Línea	Qué hace
`lyrNorm`	1118	Normaliza texto de letras para buscar (sin acentos ni signos).
`LYRX.load`	1121	Carga el índice de letras guardado.
`LYRX.add`	1130	Agrega la letra de una canción al índice.
`LYRX.save`	1131	Guarda el índice de letras cifrado (con espera).
`LYRX.reset`	1139	Borra el índice de letras.
`LYRX.build`	1141	Arma o completa el índice leyendo letras de a 3.
`LYRX.search`	1167	Busca canciones cuya letra contiene un texto.
`lyrSnippetHTML`	1184	Línea de letra con lo buscado resaltado (escapada).
`lyricsHitsHTML`	1189	Lista HTML de canciones encontradas por la letra.
`lyricsSection`	1194	Sección «En las letras» que se completa mientras se indexa.

Ayudantes de dibujo desde la línea 1210 · 8

Función	Línea	Qué hace
`coverHTML`	1212	HTML de una carátula con su ícono de respaldo.
`head`	1217	Encabezado de una pantalla (título, botones, oso, atrás).
`albumCard`	1221	Tarjeta de un álbum.
`artistCard`	1222	Tarjeta de un artista.
`trackRows`	1223	Contenedor de filas de canciones.
`rowsHTML`	1224	Filas de canciones (número, título, duración, menú).
`loading`	1236	Indicador de carga.
`errBox`	1237	Caja de error con «Reintentar».

Vistas desde la línea 1240 · 6

Función	Línea	Qué hace
`V.albums`	1243	Pantalla Álbumes (por etiquetas, de 120 en 120).

Función	Línea	Qué hace
V.album	1258	Pantalla de un álbum.
normTxt	1289	Normaliza texto (sin acentos, minúsculas).
mixDef	1290	Definición de una mezcla (consulta según géneros o años).
mixTracks	1304	Canciones de una mezcla (en caché).
greeting	1305	Saludo según la hora.

DJ Nanuq desde la línea 1307 · 85

Función	Línea	Qué hace
djBearSVG	1313	SVG de DJ Nanuq con audífonos, bufanda y tornamesa.
djMoment	1365	Momento del día y ánimo para la ronda «Para este momento».
pick	1366	Elige uno al azar.
wpick	1368	Elige al azar con pesos.
wshuffle	1370	Ordena al azar dando más chance a lo que pesa más.
djQid	1373	Id de cola que lleva la sesión y la ronda del DJ.
djTag	1374	Lee sesión y ronda de un id de cola del DJ.
djSame	1375	¿Dos ids son de la misma sesión del DJ?
DJ.voiceOk	1378	¿Hay una voz del sistema en el idioma de la app?
DJ.save	1379	Guarda la sesión del DJ y la comparte con tus dispositivos.
DJ.restore	1384	Recupera la sesión del DJ al abrir la app.
DJ.sample	1391	Muestra al azar de la biblioteca para elegir géneros y décadas.
DJ.heavy	1397	Tus 60 canciones más escuchadas.
DJ.pickGenre	1403	Elige el género de una ronda (sin repetir los recientes).
DJ.pickDecade	1413	Elige la década de una ronda.
DJ.rank	1425	Qué tan gastada está una canción (más bajo = va antes).
DJ.pool	1434	Canciones candidatas para un tipo de ronda.
DJ.build	1467	Arma una ronda de ~5 canciones con su presentación.
DJ.next	1494	Elige y arma la siguiente ronda (pesos y enfriamiento).
DJ.items	1508	Convierte una ronda en canciones de la cola y la anota.
DJ.lastKey	1516	Número de la última ronda.
DJ.start	1517	Empieza una sesión del DJ con su pantalla de carga (aquí o en el dispositivo que controlas).
DJ.startNow	1522	Arma la primera ronda y espera a que suene (detrás de la pantalla de carga).
DJ.share	1547	Deja en Jellyfin el estado del DJ para otro dispositivo.

Función	Línea	Qué hace
DJ.handedOff	1552	La sesión pasó a otro dispositivo: aquí se apaga.
DJ.adopt	1554	Sigue aquí una sesión que viene de otro dispositivo.
DJ.queueFromShare	1559	Reconstruye la cola del DJ desde el estado compartido.
DJ.queueFromTags	1561	Reconstruye la cola del DJ desde los ids de la cola.
DJ.remoteTag	1563	¿Lo que suena en el dispositivo que controlas es el DJ?
DJ.shown	1564	¿Hay que mostrar el DJ?
DJ.remoteTrack	1566	Trae las rondas del DJ que suena en otro dispositivo.
DJ.stop	1576	Apaga el DJ (la cola queda como está).
DJ.onTrack	1578	Al cambiar de canción: presenta la ronda nueva y agrega la siguiente.
DJ.extend	1586	Agrega la siguiente ronda al final de la cola.
DJ.switchUp	1596	«Cambiar de onda»: otra ronda en lugar de lo que quedaba.
DJ.announce	1604	Presenta una ronda (globo, aviso y voz).
DJ.show	1606	Muestra la presentación donde corresponda.
DJ.speak	1611	Lee la presentación en voz alta y baja la música.
DJ.visible	1629	¿La página está a la vista?
DJ.bears	1630	Enciende o apaga el baile de los DJ.
DJ.say	1634	Globo de diálogo del DJ en el reproductor.
DJ.scene	1639	Escena del disco en el reproductor (funda, vinilo, entradas y salidas).
DJ.paint	1658	Actualiza todo lo del DJ en pantalla.
DJSHARE.ok	1688	¿Se puede compartir el DJ (Jellyfin y sincronización activada)?
DJSHARE.state	1689	Estado del DJ para compartir (acotado).
DJSHARE.push	1694	Guarda el estado del DJ en /DisplayPreferences.
DJSHARE.pushRaw	1702	Guarda un estado ya armado.
DJSHARE.pushSoon	1706	Guarda el estado en 1.5 s.
DJSHARE.offSoon	1708	Avisa que el DJ se apagó, si la sesión sigue siendo de aquí.
DJSHARE.pull	1712	Lee el estado compartido del DJ.
djClean	1718	Limpia el estado del DJ leído (dato no confiable).
djLiveSync	1733	Anima el DJ solo mientras se ve.
djTileHTML	1745	Tarjeta de DJ Nanuq en Inicio.
DJL.show	1755	Muestra la pantalla de carga del DJ con frases que rotan.
DJL.hide	1765	Quita la pantalla de carga (al menos 1 s a la vista).
V.dj	1770	Pantalla de DJ Nanuq.

Función	Línea	Qué hace
nanuqSceneSVG	1794	Escena de Nanuq para una celda vacía de Inicio.
nanuqBear	1810	Nanuq listo para colocar en una escena.
nanuqWideSVG	1814	Escena ancha de Nanuq (dos celdas).
nanuqWideTileHTML	1831	Celda ancha con escena.
nanuqTileHTML	1836	Celda con escena.
nanuqVisit	1843	Un Nanuq de otro tema visita una celda.
nanuqVisits	1863	Programa visitas de vez en cuando.
fillGaps	1871	Rellena los huecos de la cuadrícula de Inicio con escenas.
tileHTML	1884	Mosaico de una mezcla.
offlineBanner	1889	Aviso «Sin conexión» con acceso a Descargas.
V.home	1894	Pantalla Inicio.
V.mix	1939	Pantalla de una mezcla.
V.songs	1949	Pantalla Canciones (paginada).
V.artists	1977	Pantalla Artistas.
fmtLong	1984	Duración larga («3 h 12 min»).
V.artist	1986	Pantalla de un artista (canciones, álbumes, aparece en).
V.genres	2066	Pantalla Géneros.
V.genre	2071	Pantalla de un género.
V.playlists	2082	Pantalla Playlists.
V.playlist	2087	Pantalla de una playlist.
V.favorites	2096	Pantalla Favoritas (confirmada con Jellyfin).
V.folders	2122	Pantalla Carpetas.
V.search	2131	Pantalla Buscar (incluye la búsqueda en letras).
songsBlock	2161	Bloque de canciones en lista o cuadrícula.
V.queue	2168	Pestaña Cola (teléfono).
V.more	2169	Pantalla Más.
V.settings	2172	Pantalla Ajustes.
V.downloads	2241	Pantalla Descargas.
renderDownloadsSoon	2257	Redibuja Descargas en un momento.

Tu año y Tu semana (video) desde la línea 2263 · 37

Función	Línea	Qué hace
History.purge	2271	Borra el historial local de versiones anteriores.
ServerStats.available	2281	¿Está el plugin Playback Reporting y se puede leer?

Función	Línea	Qué hace
`ServerStats.plays`	2289	Reproducciones entre dos fechas desde el plugin (día por día).
`Recap.total`	2345	Duración total del video.
`Recap.setMode`	2346	Cambia entre resumen del año y de la semana.
`Recap.plays`	2351	Reproducciones del periodo (plugin, datos básicos o demo).
`Recap.tally`	2368	Agrupa reproducciones: canciones, artistas, géneros, horas, días, racha.
`Recap.collect`	2392	Datos del resumen del año.
`Recap.collectWeek`	2404	Datos del resumen de la semana pasada (lunes a domingo).
`Recap.jellyfinYear`	2420	Canciones escuchadas en un periodo según los datos básicos de Jellyfin.
`Recap.assets`	2433	Prepara imágenes del video: osos, DJ, carátulas y paleta.
`Recap.loadImg`	2455	Carga una imagen con topes (8 MB, reducida) para el canvas.
`Recap.sceneAt`	2469	Qué escena toca en un segundo del video.
`Recap.frame`	2470	Dibuja un cuadro del video (con transición entre escenas).
`Recap.transition`	2479	Transición entre dos escenas (deslizar, círculo, zoom, barrido).
`Recap.bg`	2489	Fondo de una escena: degradado, brillo al ritmo y copos.
`Recap.weather`	2506	Dibuja el pronóstico musical (sol, tormenta, nieve, arcoíris…).
`Recap.bear`	2523	Dibuja un Nanuq que brinca al ritmo.
`Recap.text`	2530	Texto ajustado al ancho, con sombra.
`Recap.cover`	2540	Carátula con esquinas redondeadas.
`Recap.confetti`	2546	Lluvia de confeti.
`Recap.scene`	2554	Dibuja una escena del resumen.
`Recap.music`	2801	Música del video generada con Web Audio (con redoble).
`Recap.stop`	2830	Detiene la reproducción o grabación del video.
`Recap.play`	2831	Reproduce el video en el canvas y, si se pide, lo graba (MediaRecorder).
`Recap.save`	2859	Guarda el video (Android en trozos, escritorio como descarga).
`mp4Finalize`	2885	Escribe la duración real en el MP4 fragmentado.
`recapCloseAudio`	2926	Cierra el audio del video una sola vez.
`ease.out`	2927	Curva de animación (salida).
`ease.inOut`	2927	Curva de animación (entrada y salida).
`ease.back`	2927	Curva de animación con rebote.
`clamp01`	2928	Limita un valor entre 0 y 1.

Función	Línea	Qué hace
mixHex	2929	Mezcla dos colores hex.
hexA	2930	Color hex con transparencia.
recapYearOpen	2932	¿Es la primera semana de diciembre?
recapTileHTML	2933	Tarjeta «Tu año con Nanuq» en Inicio.
V.recap	2936	Pantalla de Tu semana / Tu año (ver y exportar).

Solicitar canción (SomeDL) desde la línea 2979 · 26

Función	Línea	Qué hace
__nanuqHttp	2985	Recibe la respuesta de una petición nativa a SomeDL (Android).
reqNative	2986	¿Hay puente nativo para hablar con SomeDL sin CORS?
reqRawHttp	2987	Petición a SomeDL por el puente nativo o fetch.
parseSomedl	2997	Interpreta la dirección de SomeDL (puerto 5000 por omisión).
reqNetError	3006	Mensaje claro cuando SomeDL no responde.
REQ.save	3021	Guarda la lista de pedidos.
REQ.setCfg	3022	Guarda o quita la dirección de SomeDL.
REQ.pending	3023	Pedidos que siguen en camino.
REQ.visible	3024	¿La app está a la vista (para revisar más seguido)?
REQ.http	3025	Petición a la API de SomeDL con manejo de errores.
REQ.probe	3033	Comprueba que una dirección sea SomeDL.
REQ.add	3039	Manda un pedido de canción a SomeDL.
REQ.schedule	3051	Programa la siguiente revisión de pedidos.
REQ.poll	3052	Revisa el estado de los pedidos en SomeDL.
REQ.announce	3081	Avisa (y notifica) qué pedidos terminaron.
REQ.retry	3096	Vuelve a pedir una canción que falló.
sysNotify	3103	Notificación del sistema (Android o escritorio).
askNotifyPermission	3109	Pide permiso de notificaciones en el navegador.
reqRowHTML	3112	Fila de un pedido de SomeDL.
reqListHTML	3121	Lista de pedidos.
renderReqSoon	3128	Redibuja la lista de pedidos en un momento.
V.request	3132	Pantalla Solicitar canción.
refreshFromJellyfin	3192	«Actualizar desde Jellyfin»: olvida cachés, rehace índices y refresca la cola.
sessionGone	3215	Confirma si un 401 es de verdad la sesión vencida.

Función	Línea	Qué hace
sessionExpired	3226	Sesión vencida: detiene todo y pide entrar.
render	3234	Dibuja la pantalla actual con transición y manejo de errores.

Reproductor desde la línea 3270 · 61

Función	Línea	Qué hace
P.pause	3280	Pausa marcando que la pidió Nanuq.
onAudio	3285	Escucha un evento en los dos <audio> pero solo atiende al de la canción actual.
saveQueue	3287	Guarda la cola y actualiza la ventana por adelantado.
QPOS.save	3292	Guarda el segundo de la canción para retomarlo.
QPOS.restore	3298	Dónde retomar la canción al abrir la app.
QPOS.clear	3302	Olvida la posición guardada.
setQueue	3306	Reemplaza la cola y empieza a reproducir.
addToQueue	3307	Agrega canciones al final o a continuación.
ptime	3317	Segundo actual de la canción (aquí o en el otro dispositivo).
pdur	3318	Duración de la canción.
setSrc	3324	Cambia la fuente del audio reiniciando su estado.
seekTo	3328	Salta a un segundo (en conversiones en vivo pide desde ahí).
playIndex	3349	Reproduce la canción N de la cola.
armStallWatch	3380	Avisa si una canción tarda demasiado en empezar.
MP3.add	3394	Recuerda una canción que debe pedirse en MP3.
localFailed	3398	La copia local falló: suena del servidor (y se vuelve a descargar).
canFallback	3410	¿Se puede pedir esta canción en MP3 (en «Original» o con Opus)?
fallbackMp3	3413	Pide la canción en MP3 si la original o la de Opus no arranca.
armFallback	3423	Programa el paso a MP3 si no arranca en 6 s.
playFailed	3429	Explica por qué no se pudo reproducir.
audioDead	3450	¿El audio quedó muerto (error de red)?
replayHere	3451	Vuelve a pedir la canción desde el segundo en que iba.
rampStop	3464	Detiene una rampa de volumen.
ramp	3468	Rampa de volumen (en decibelios, o de potencia constante para el crossfade) con temporizador.
fadeStop	3477	Detiene un desvanecido en curso.
fadeTo	3478	Lleva el volumen a un valor con desvanecido.

Función	Línea	Qué hace
softMs	3479	Duración del desvanecido de pausa/play (0 si está apagado).
userVol	3483	Volumen elegido por ti (bajado si habla el DJ).
pauseSoft	3484	Pausa con desvanecido.
playSoft	3492	Reanuda con desvanecido, o vuelve a subir si estaba bajando (recarga si otra app pausó).
partyNow	3502	¿Los controles manejan la fiesta ahora?
partyHostOpen	3505	¿Hay una sala de fiesta abierta en esta computadora?
toggle	3506	Reproducir/pausar (fiesta, control remoto o aquí).
next	3515	Siguiente canción (respeta repetir y el fin de la cola).
prev	3527	Canción anterior (o al inicio si van más de 3 s).
seekFrac	3528	Salta a una fracción de la canción.
removeAt	3529	Quita una canción de la cola.
clearQueue	3534	Vacía la cola.
shuffleQueue	3535	Mezcla la cola (la actual queda primero).
setMode	3543	Cambia el modo de repetición.
xfDrop	3551	Libera el <audio> de la canción que se fue.
xfKill	3553	Apaga la canción que se iba en la transición (al pausar o cambiar de canción).
xfCheck	3557	Decide si empieza la transición del DJ (a 3–5 s del final).
xfStart	3566	Cruza la canción que termina con la siguiente usando los dos <audio>.
updateModeBtn	3580	Actualiza el botón de repetición.
loadPeaks	3582	Carga la forma de onda de una canción.
cssVar	3591	Lee una variable CSS del tema (en caché).
drawWaves	3592	Dibuja las barras de progreso o la forma de onda.
roundRect	3620	Rectángulo redondeado en canvas.
updateNow	3622	Actualiza todo lo que muestra la canción actual.
updatePlayState	3652	Íconos de reproducir/pausa y estado de los controles del sistema.
DISC.can	3667	¿Hay puente de Discord (escritorio)?
DISC.update	3668	Programa el aviso a Discord.
DISC.send	3669	Manda a Discord lo que suena (o lo quita).
dcStatusPaint	3685	Línea de estado de Discord en Ajustes.
rcKeep	3706	Audio en silencio para que el sistema mantenga los controles.

Función	Línea	Qué hace
sessionHold	3720	Mantiene los controles del sistema mientras carga o al terminar la cola.
rcKeepOff	3726	Detiene el audio en silencio.
stallCheck	3766	Detecta audio atorado a media canción y lo vuelve a pedir.
netBack	3775	Volvió la red: reanuda lo cortado y reconecta.
paintVol	3818	Dibuja la barra de volumen y su ícono.

Letras desde la línea 3823 · 32

Función	Línea	Qué hace
lrcKey	3826	Clave de una letra guardada en el dispositivo.
lrcTime	3827	Formatea un tiempo como mm:ss.cc (LRC).
parseLRC	3828	Lee un archivo LRC (marcas de tiempo y offset).
toLRC	3842	Arma un archivo LRC limpio.
cleanLyricsText	3853	Limpia una letra escrita o pegada (controles, HTML, tamaño).
lyricsFromText	3862	Convierte texto (LRC o simple) en letra.
localLyrics	3863	Letra guardada en este dispositivo.
lyrSame	3871	¿Dos letras son iguales (texto y tiempos)?
lyricsSaved	3879	Muestra la letra recién guardada hasta que Jellyfin la sirva.
getLyrics	3894	Letra de una canción (local, pendiente, caché o servidor).
setLyricsMode	3902	Abre o cierra la letra con transición.
setQueueMode	3915	Abre o cierra la cola del reproductor con transición.
renderSheetQueue	3926	Lista de la cola dentro del reproductor.
applyLyricsMode	3935	Aplica el modo letra sin transición.
reloadLyrics	3943	Vuelve a cargar la letra actual.
renderLyrics	3944	Dibuja la letra de la canción actual.
estimateTimes	3968	Reparte tiempos aproximados para letras sin sincronizar.
syncLyrics	3975	Resalta la línea que suena y desplaza la letra.
refreshLyricsButtons	4011	Botón Letra o Buscar letra según haya letra.
syncStart	4026	Abre el editor para sincronizar a mano.
syncEnd	4040	Cierra el editor.
syncCancel	4044	Cancela la sincronización.
renderSync	4045	Dibuja el editor de sincronización.
syncEditorTick	4052	Marca en el editor la línea que suena.
syncMark	4059	Marca el inicio de la línea resaltada.
syncUndo	4072	Deshace la última marca.

Función	Línea	Qué hace
syncNudge	4080	Mueve la última marca ¼ s.
syncSave	4089	Completa los tiempos y guarda el LRC.
saveLyricsText	4106	Guarda una letra en Jellyfin (o en el dispositivo si no se puede).
lyricsSearch	4118	Busca letras en LrcLib y deja elegir una.
lyricsPaste	4148	Diálogo para pegar o editar una letra.
forgetLocalLyrics	4166	Quita la letra guardada en este dispositivo.

Cola desde la línea 4169 · 4

Función	Línea	Qué hace
renderQueue	4171	Dibuja el panel de la cola.
renderQueueAll	4180	Redibuja todas las vistas de la cola.
moveQueue	4185	Mueve una canción de la cola (también en el control remoto).
markCurrent	4237	Resalta la canción que suena en las listas.

Avisos, menús y diálogos desde la línea 4240 · 2

Función	Línea	Qué hace
toast	4242	Aviso breve en pantalla.
menu	4243	Menú de opciones (hoja inferior).

Mis dispositivos desde la línea 4251 · 63

Función	Línea	Qué hace
devLabel	4256	Nombre amable de un dispositivo.
devName	4258	Nombre de un dispositivo para frases.
expand6	4260	Expande una dirección IPv6.
ipKey	4266	Red y privacidad de una dirección IP.
sameNetwork	4274	¿Dos direcciones están en la misma red?
RC.on	4286	¿Mis dispositivos está activado?
RC.start	4287	Arranca Mis dispositivos (WebSocket y búsqueda).
RC.stop	4288	Detiene Mis dispositivos.
RC.connect	4294	Abre el WebSocket de Jellyfin con reintentos.
RC.send	4307	Manda un mensaje por el WebSocket.
RC.onMessage	4308	Atiende los mensajes del WebSocket (órdenes, favoritas, keep-alive).
RC.fromMe	4319	¿La orden viene de tu propio usuario?

Función	Línea	Qué hace
RC.onPlaystate	4326	Ejecuta pausar, siguiente, buscar… recibidos.
RC.onPlay	4339	Ejecuta una orden de reproducir (y adopta el DJ si viene con él).
RC.onGeneral	4361	Ejecuta repetir o mezclar recibidos.
RC.queueForReport	4368	Cola que se informa a Jellyfin.
RC.reportSoon	4369	Informa el estado a Jellyfin en un momento.
RC.schedule	4375	Programa la siguiente búsqueda de dispositivos.
RC.poll	4376	Busca tus otros Nanuq y sigue al que controlas.
RC.maybeFollow	4399	Se conecta solo a otro Nanuq que suena en tu red.
RC.enter	4405	Empieza a controlar otro dispositivo.
RC.leave	4421	Deja de controlar y recupera tu cola.
RC.apply	4429	Refleja aquí lo que suena en el otro dispositivo.
RC.pos	4460	Segundo estimado de la canción en el otro dispositivo.
RC.tick	4461	Actualiza barra, letra y controles del sistema en control remoto.
RC.decorate	4468	Indica «Sonando en …» en el reproductor.
RC.fail	4474	Avisa que una orden no se pudo enviar.
RC.cmd	4475	Manda una orden al dispositivo que suena.
RC.seek	4481	Salta en el otro dispositivo.
RC.window	4482	Ventana de la cola que se manda (≤ 200).
RC.playQueue	4483	Manda una cola para reproducir allá.
RC.add	4492	Agrega canciones a la cola de allá.
RC.reorder	4502	Manda la cola reordenada (conservando el DJ).
RC.general	4513	Manda una orden general allá.
RC.here	4515	«Reproducir aquí»: trae la música (y el DJ) a este dispositivo.
RC.there	4538	Pasa la música (y el DJ) a otro dispositivo.
__nanuqTick	4557	Latido cada 20 s desde Android (WebSocket, fiesta, audio atorado).
__nanuqVis	4565	Android avisa si la app está a la vista.
openDevices	4566	Menú Mis dispositivos.
updateFavBtn	4597	Botón de favorita del reproductor.
setFavLocal	4603	Marca la favorita en la cola local.
favPending	4610	Recuerda un cambio de favorita reciente.
favMerge	4611	Aplica los cambios recientes sobre la lista del servidor.

Función	Línea	Qué hace
favHold	4629	Protege la favorita de la canción que suena (fallo de Jellyfin 12).
favHeld	4632	¿Se está protegiendo esta favorita?
favReassert	4633	Vuelve a guardar la favorita después de cada aviso de reproducción.
favRemote	4641	Atiende cambios de favoritas hechos en otro lado.
refreshFav	4663	Confirma con el servidor si la canción es favorita.
toggleFav	4674	Marca o quita la favorita y lo comprueba.
favConfirm	4694	Segunda confirmación contra la sección de favoritas.
updateDlBtn	4708	Botón Descargar del reproductor.
dlAddToast	4717	Aviso al pedir una descarga.
dlTap	4726	Toque en Descargar (pedir o administrar).
connTest	4739	«Probar conexión»: mide API, carátula, audio, saltos y reproductor.
startRadio	4867	Radio a partir de una canción.
addToPlaylistMenu	4881	Menú para agregar a una playlist (o crear una).
songOptions	4896	Opciones comunes de una canción.
queueMenu	4903	Menú de una canción de la cola.
trackMenu	4916	Menú de una canción de una lista.
askConfirm	4924	Diálogo de confirmación.
askName	4930	Diálogo para escribir un nombre.
openSheet	4938	Abre el reproductor maximizado.
closeSheet	4946	Cierra el reproductor maximizado.

Fiesta (watch party) desde la línea 4949 · 65

Función	Línea	Qué hace
partyHost	4956	¿Este dispositivo puede ser anfitrión (escritorio)?
partyGuest	4957	¿Este dispositivo es invitado (Android)?
partyOn	4958	¿Hay fiesta en este dispositivo?
partySongUrl	4960	URL en MP3 320 kbps para mandar una canción a la fiesta.
partyCover	4966	Miniatura JPEG de 240 px de la carátula.
partyCoverOk	4976	¿La carátula viene de la sala (127.0.0.1 con token)?
partyClean	4979	Limpia el estado que manda la computadora (no confiable).
stopForParty	4991	Detiene tu música al abrir la sala.
PH.active	5006	¿La fiesta maneja los controles?

Función	Línea	Qué hace
PH.init	5007	Prepara el reproductor de la fiesta y escucha a main.js.
PH.onEv	5019	Atiende los eventos de la sala (estado, llegó, salió).
PH.apply	5025	Aplica el estado de la sala (modo fiesta de la computadora).
PH.list	5037	Lista de canciones de la sala.
PH.open	5038	Abre la sala.
PH.close	5044	Cierra la sala.
PH.jump	5046	Pone ya una canción de «A continuación».
PH.order	5052	Cambia el orden de la lista.
PH.next	5054	Sigue la lista: pone la siguiente que ya llegó.
PH.play	5061	Reproduce una canción de la fiesta.
PH.stopCur	5069	Detiene la canción actual de la fiesta.
PH.done	5077	Terminó una canción: la borra y pasa a la siguiente.
PH.toggle	5078	Pausa o reanuda la fiesta.
PH.yieldTo	5091	Pusiste tu música: la fiesta se pausa y cede los controles.
PH.media	5098	Controles del sistema con la canción de la fiesta.
PH.report	5110	Avisa a la sala qué suena.
PH.loadLyrics	5115	Carga la letra de la canción de la fiesta.
PH.line	5125	Línea de letra actual y la siguiente.
PH.tick	5133	Karaoke: cada segundo, tiempo y línea a los teléfonos.
PG.connect	5143	Entra a una sala (con código o llave) y reconecta solo.
PG.leave	5170	Sale de la fiesta.
PG.joined	5180	¿Estás en una fiesta?
PG.ui	5181	Modo fiesta del teléfono (oculta barras y reproductor).
PG.wake	5190	El teléfono despertó: vuelve a entrar si hace falta.
PG.resume	5196	Al abrir la app, vuelve a la fiesta en curso.
PG.abortUp	5201	Cancela la subida en curso.
PG.onMsg	5202	Atiende los mensajes de la sala.
PG.add	5229	Manda una canción a la fiesta (letra, carátula y audio por partes).
PG.remove	5285	Cancela una canción que todavía llega.
PG.find	5287	Busca salas en la red (UDP desde Android).
partyAddr	5307	Convierte la dirección escrita en ws://.../fiesta.
iglooHTML	5318	HTML del iglú.
iglooBear	5334	Nanuq de un invitado con el disfraz de su tema.

Función	Línea	Qué hace
iglooHoldSVG	5341	Carátula en alto con destellos (quien escogió la canción).
iglooHeldOnTop	5352	Pone encima de la carátula el objeto que traía el Nanuq.
partyGuestColor	5363	Color de la etiqueta según el tema del invitado.
partyCoverSrc	5371	URL de la carátula de una canción de la fiesta.
iglooSync	5379	Coloca a los invitados en el círculo sin redibujar.
iglooLive	5413	Anima el iglú solo mientras se ve.
iglooQueue	5418	Fila «A continuación» a pantalla completa.
iglooQueueEdge	5430	Degradado si hay más canciones fuera de la vista.
partyVolumeStep	5506	Volumen con ↑ ↓ a pantalla completa y su aviso.
iglooIdle	5519	Esconde el puntero tras 2.5 s quieto.
iglooVinyl	5529	Funda y disco de vinilo de la canción que suena.
iglooLine	5541	Letra en el techo con destello.
renderPartySoon	5548	Redibuja la fiesta en un momento.
partyPaintTick	5552	Actualiza el karaoke (línea, siguiente, tiempo).
partyPaintUp	5558	Progreso de la canción que mandas.
partyRowsHTML	5559	Lista de canciones de la fiesta.
partyGuestsHTML	5568	Lista de invitados.
partyNowHTML	5569	Lo que suena en la fiesta.
paintParty	5574	Dibuja la pantalla de la fiesta (anfitrión o invitado).
partySongsHTML	5635	Canciones para mandar.
partySearch	5636	Buscador de la fiesta (playlists, artistas, álbumes, canciones, letras).
partyBrowse	5664	Canciones de un artista, álbum o playlist en la fiesta.
V.party	5679	Pantalla Fiesta.

Acciones globales desde la línea 5693 · 1

Función	Línea	Qué hace
d0	5695	Atajo del dataset de un elemento.

Entrar desde la línea 5853 · 11

Función	Línea	Qué hace
recentServers	5856	Servidores usados antes.
rememberServer	5857	Recuerda un servidor.
forgetServer	5863	Olvida un servidor.
probeTimed	5865	Prueba un servidor midiendo cuánto tarda.

Función	Línea	Qué hace
`canDiscover`	5876	¿Se pueden buscar servidores en la red?
`discoverServers`	5878	Busca Jellyfin en la red (UDP 7359).
`serverList`	5897	Lista de servidores con su estado.
`cleartextRisk`	5924	¿La contraseña viajaría sin cifrar por internet?
`showLogin`	5935	Pantalla de entrar.
`startApp`	6001	Arranca la app con la sesión (cola, posición, DJ, Inicio).
`logout`	6017	Cierra sesión y borra lo personal.

17 Resto de la página

Idiomas y service worker.



web/i18n.js

1024 líneas · 1 funciones

Función	Línea	Qué hace
tx	1015	Traduce un texto en español al idioma elegido y pone los valores {0}, {1}...

web/sw.js

45 líneas · 3 funciones

Función	Línea	Qué hace
install	8	Guarda en caché los archivos de la app al instalar el service worker.
activate	12	Borra las cachés de versiones anteriores.
fetch	16	Sirve solo los archivos de la app (red primero para HTML/JS/CSS); la API y la música van directo a la red.

18 Android

Java nativo y el puente de JavaScript.



android/app/src/main/java/com/nanuq/app/Discovery.java

78 líneas · 3 funciones

Función	Línea	Qué hace
`Discovery.Discovery`	27	Constructor privado (solo métodos estáticos).
`Discovery.find`	30	Busca por UDP (Jellyfin o fiestas) y devuelve las respuestas en JSON.
`Discovery.send`	72	Manda la pregunta a todas las direcciones de difusión.

android/app/src/main/java/com/nanuq/app/Downloads.java

595 líneas · 44 funciones

Función	Línea	Qué hace
`Downloads.get`	43	Instancia única de las descargas.
`Downloads.Downloads`	72	Prepara carpeta, índice, servidor local y vigilancia de red.
`Downloads.audio`	83	Archivo de audio para el servidor local.
`Downloads.audioMime`	84	Tipo del audio para el servidor local.
`Downloads.cover`	85	Carátula para el servidor local.
`Downloads.prefetch`	86	Canción guardada por adelantado para el servidor local.
`Downloads.prefetchMime`	87	Tipo de la canción guardada por adelantado.
`Downloads.onAvailable`	91	Volvió la red: reintenta las descargas.
`Downloads.onCapabilitiesChanged`	92	Cambió la red: revisa la cola de descargas.
`Downloads.setListener`	97	Conecta la página para recibir cambios.
`Downloads.configure`	101	Máximo de canciones y solo Wi-Fi.
`Downloads.base`	117	Dirección del servidor local.
`Downloads.okUrl`	127	¿La URL tiene la forma de audio o carátula de Jellyfin?
`Downloads.openSameHost`	141	Abre una conexión siguiendo redirecciones solo al mismo servidor.
`Downloads.enqueue`	162	Agrega una descarga (o la actualiza).
`Downloads.setScore`	196	Actualiza cuántas veces sonó una descarga.
`Downloads.remove`	202	Quita una descarga.
`Downloads.removeAll`	208	Quita todas (o solo las automáticas).
`Downloads.list`	214	Lista de descargas en JSON.

Función	Línea	Qué hace
`Downloads.network`	222	Red actual (wifi, datos, ninguna), mirando debajo de una VPN.
`Downloads.underlying`	236	Red real debajo de una VPN.
`Downloads.file`	245	Archivo de una descarga lista.
`Downloads.coverFile`	253	Carátula de una descarga.
`Downloads.setCover`	262	Pide la carátula de una descarga vieja.
`Downloads.fetchCoverLater`	270	Baja la carátula en segundo plano.
`Downloads.fetchCover`	273	Baja la carátula (≤ 15 MB).
`Downloads.mime`	301	Tipo de audio de una descarga.
`Downloads.retryAndKick`	309	Reintenta las fallidas y sigue.
`Downloads.kick`	316	Arranca la cola de descargas.
`Downloads.kickLocked`	318	Arranca la cola (ya con el candado).
`Downloads.loop`	324	Baja las pendientes de una en una.
`Downloads.nextPending`	344	La siguiente (primero las pedidas por ti).
`Downloads.download`	354	Baja una canción con topes y espacio libre; Opus solo si está completo (OggCheck).
`Downloads.victim`	431	La automática menos escuchada (para hacer lugar).
`Downloads.removeEntry`	440	Borra una descarga y sus archivos.
`Downloads.caps`	454	Capacidades de la red activa.
`Downloads.unmetered`	459	¿La red es sin medición (Wi-Fi, Ethernet)?
`Downloads.networkAllowed`	464	¿Se puede descargar con esta red?
`Downloads.safe`	472	Nombre de archivo seguro para un id.
`Downloads.toJson`	474	Descarga en JSON para la página.
`Downloads.emit`	492	Avisa a la página un cambio.
`Downloads.ensureLoaded`	498	Lee el índice cuando el almacén se abre.
`Downloads.load`	500	Lee el índice cifrado (o recupera desde los archivos).
`Downloads.save`	566	Guarda el índice cifrado.

`android/app/src/main/java/com/nanuq/app/LocalServer.java` 184 líneas · 7 funciones

Función	Línea	Qué hace
`LocalServer.LocalServer`	46	Abre el servidor en 127.0.0.1 con token aleatorio.
`LocalServer.base`	60	Dirección base con el token.
`LocalServer.acceptLoop`	62	Acepta conexiones (máx. 32 a la vez).
`LocalServer.readLine`	74	Lee una línea de la petición con tope y hora límite.
`LocalServer.handle`	86	Atiende una petición (10 s y ≤ 100 encabezados).

Función	Línea	Qué hace
`LocalServer.status`	129	Responde un código de estado.
`LocalServer.serveFile`	135	Entrega un archivo con rangos (saltar dentro de la canción).

android/app/src/main/java/com/nanuq/app/MainActivity.java

406 líneas · 44 funciones

Función	Línea	Qué hace
`MainActivity.run`	63	Latido cada 20 s hacia la página.
`MainActivity.onCreate`	70	Crea el WebView, conecta servicios y carga la app.
`MainActivity.createWeb`	88	Configura el WebView (sin acceso a archivos, puente).
`MainActivity.recreateWeb`	114	Rehace el WebView si el sistema lo cerró.
`MainActivity.dispatchMedia`	124	Pasa un botón de la notificación a la página.
`MainActivity.onBackPressed`	135	Atrás: la página decide o la app va al fondo.
`MainActivity.onStart`	144	La app volvió a la vista.
`MainActivity.onStop`	147	La app pasó al fondo (guarda el almacén).
`MainActivity.onDestroy`	150	Libera el WebView y la notificación.
`MainActivity.setBarColor`	164	Color de las barras del sistema según el tema.
`MainActivity.showNotice`	185	Notificación de «canción solicitada lista».
`MainActivity.setMetadata`	213	Puente: título, artista, álbum y carátula.
`MainActivity.setPlaybackState`	214	Puente: reproduciendo o en pausa.
`MainActivity.setActions`	215	Puente: botones disponibles.
`MainActivity.setPosition`	216	Puente: posición y duración.
`MainActivity.dlList`	224	Puente: lista de descargas.
`MainActivity.dlEnqueue`	225	Puente: descargar una canción.
`MainActivity.dlCover`	226	Puente: pedir una carátula.
`MainActivity.dlBase`	227	Puente: dirección del servidor local.
`MainActivity.dlRemove`	228	Puente: quitar una descarga.
`MainActivity.dlRemoveAll`	229	Puente: quitar descargas.
`MainActivity.dlConfig`	230	Puente: ajustes de descargas.
`MainActivity.dlScore`	231	Puente: reproducciones de una descarga.
`MainActivity.dlNetwork`	232	Puente: tipo de red.
`MainActivity.pfSet`	235	Puente: ventana de lo que sigue por adelantado.
`MainActivity.pfReady`	236	Puente: ¿ya está guardada esa canción?
`MainActivity.pfBroken`	237	Puente: esa canción guardada no sonó.
`MainActivity.http`	240	Puente: petición a SomeDL sin CORS.
`MainActivity.secKind`	248	Puente: estado del almacén cifrado.

Función	Línea	Qué hace
`MainActivity.secGet`	249	Puente: leer un dato cifrado.
`MainActivity.secSet`	250	Puente: guardar un dato cifrado.
`MainActivity.secDel`	251	Puente: borrar un dato cifrado.
`MainActivity.secReset`	253	Puente: empezar de cero.
`MainActivity.videoBegin`	257	Puente: empieza a guardar un video (MP4/WebM).
`MainActivity.videoChunk`	269	Puente: un trozo del video (≤ 300 MB).
`MainActivity.videoEnd`	281	Puente: termina y guarda en Películas › Nanuq.
`MainActivity.videoShare`	323	Puente: compartir el video recién guardado.
`MainActivity.notice`	332	Puente: notificación del sistema.
`MainActivity.partyDiscover`	335	Puente: buscar fiestas por UDP.
`MainActivity.discover`	343	Puente: buscar Jellyfin por UDP.
`MainActivity.onRenderProcessGone`	354	El proceso del WebView murió: se rehace.
`MainActivity.shouldInterceptRequest`	360	Sirve la app desde los assets en http://localhost.
`MainActivity.shouldOverrideUrlLoading`	377	Solo enlaces http(s) salen al navegador.
`MainActivity.mime`	399	Tipo de un archivo de los assets.

`android/app/src/main/java/com/nanuq/app/MediaCenter.java` 372 líneas · 33 funciones

Función	Línea	Qué hace
`MediaCenter.get`	52	Instancia única del centro multimedia.
`MediaCenter.MediaCenter`	72	Crea la MediaSession y el canal de notificación.
`MediaCenter.onPlay`	83	Botón reproducir del sistema.
`MediaCenter.onPause`	84	Botón pausa del sistema.
`MediaCenter.onStop`	85	Botón detener del sistema.
`MediaCenter.onSkipToNext`	86	Botón siguiente del sistema.
`MediaCenter.onSkipToPrevious`	87	Botón anterior del sistema.
`MediaCenter.onSeekTo`	88	Saltar desde el sistema.
`MediaCenter.onFastForward`	89	Adelantar desde el sistema.
`MediaCenter.onRewind`	90	Retroceder desde el sistema.
`MediaCenter.attach`	96	Conecta la página.
`MediaCenter.detach`	98	Desconecta la página.
`MediaCenter.hasPage`	101	¿Hay página que reciba órdenes?
`MediaCenter.stopIfNoPage`	104	Sin página, quita la notificación.
`MediaCenter.dispatch`	113	Manda una acción a la página.
`MediaCenter.setMetadata`	120	Datos de la canción (y carga la carátula).

Función	Línea	Qué hace
`MediaCenter.setPlaybackState`	141	Estado de reproducción.
`MediaCenter.setActions`	152	Botones disponibles.
`MediaCenter.setPosition`	160	Posición y duración.
`MediaCenter.isPlaying`	174	¿Está sonando?
`MediaCenter.currentPosition`	176	Posición actual estimada.
`MediaCenter.publishMetadata`	181	Publica los datos en la MediaSession.
`MediaCenter.publishState`	191	Publica el estado y los botones.
`MediaCenter.refresh`	212	Ajusta servicio, notificación y bloqueos.
`MediaCenter.shutdown`	240	Detiene todo al cerrar la app.
`MediaCenter.buildNotification`	251	Arma la notificación de reproducción.
`MediaCenter.action`	280	Un botón de la notificación.
`MediaCenter.serviceIntent`	284	Intent de un botón hacia el servicio.
`MediaCenter.contentIntent`	292	Abrir la app desde la notificación.
`MediaCenter.onReceive`	300	Audífonos desconectados: pausa.
`MediaCenter.setNoisyReceiver`	305	Escucha la desconexión de audífonos.
`MediaCenter.loadBitmap`	319	Carga la carátula con topes (8 MB, reducida).
`MediaCenter.readCapped`	364	Lee con tope de bytes.

`android/app/src/main/java/com/nanuq/app/NativeHttp.java` 84 líneas · 5 funciones

Función	Línea	Qué hace
`NativeHttp.NativeHttp`	19	Constructor privado (solo métodos estáticos).
`NativeHttp.Result`	24	Respuesta (estado y texto).
`NativeHttp.allowed`	30	¿Ruta permitida de SomeDL?
`NativeHttp.fetch`	37	Petición sin CORS solo a SomeDL, sin redirecciones.
`NativeHttp.readAll`	71	Lee la respuesta (≤ 4 MB).

`android/app/src/main/java/com/nanuq/app/PlaybackService.java` 101 líneas · 7 funciones

Función	Línea	Qué hace
`PlaybackService.onCreate`	31	Prepara los bloqueos de CPU y Wi-Fi.
`PlaybackService.onStartCommand`	46	Primer plano y botones de la notificación.
`PlaybackService.setLocks`	65	Toma o suelta los bloqueos.
`PlaybackService.release`	76	Sale del primer plano y termina.
`PlaybackService.onTaskRemoved`	84	Quitada de recientes: cierra todo.
`PlaybackService.onDestroy`	92	Suelta los bloqueos.
`PlaybackService.onBind`	99	Sin enlace (servicio iniciado).

android/app/src/main/java/com/nanuq/app/Prefetch.java 222 líneas · 16 funciones

Función	Línea	Qué hace
Prefetch.get	40	Instancia única de la caché por adelantado.
Prefetch.Prefetch	61	Vacía la caché anterior y vigila la red.
Prefetch.onAvailable	71	Volvió la red: sigue bajando.
Prefetch.set	80	Recibe la ventana deseada y borra lo que sobra.
Prefetch.broken	109	Una guardada no sonó: se borra y no se repite.
Prefetch.ready	116	¿Está lista esa canción?
Prefetch.file	118	Archivo de una canción lista.
Prefetch.mime	125	Tipo de audio.
Prefetch.drop	130	Quita una canción y su archivo.
Prefetch.deleteFiles	132	Borra los archivos de una canción.
Prefetch.kick	139	Arranca las descargas.
Prefetch.kickLocked	141	Arranca (ya con el candado).
Prefetch.loop	147	Baja las pendientes de una en una (respeta el tope total).
Prefetch.online	160	¿Hay red?
Prefetch.fetch	166	Baja una canción con topes; Opus solo si está completo (OggCheck).
Prefetch.trim	211	Si se pasa del tope total, suelta las más lejanas.

android/app/src/main/java/com/nanuq/app/SecureStore.java 192 líneas · 16 funciones

Función	Línea	Qué hace
SecureStore.get	44	Instancia única (o lee un valor si recibe una clave).
SecureStore.SecureStore	58	Abre el almacén al crearse.
SecureStore.open	69	Abre clave (Keystore) y archivo: listo, locked o lost.
SecureStore.kind	104	Estado (reintenta si estaba locked).
SecureStore.reset	113	Empezar de cero: aparta el archivo y crea clave nueva.
SecureStore.ready	122	¿Está abierto?
SecureStore.set	129	Guarda un valor (≤ 8 MB).
SecureStore.del	136	Borra un valor.
SecureStore.validKey	142	¿Clave válida?
SecureStore.scheduleSave	144	Guarda en 400 ms.
SecureStore.flush	150	Guarda ya.
SecureStore.saveNow	152	Cifra y escribe el archivo.
SecureStore.encrypt	161	Cifra con AES-256-GCM.
SecureStore.decrypt	171	Descifra y verifica.

Función	Línea	Qué hace
`SecureStore.readAll`	178	Lee un archivo (≤ 64 MB).
`SecureStore.writeAtomic`	186	Escribe de forma atómica.

`android/bridge/android-bridge.js` 97 líneas · 9 funciones

Función	Línea	Qué hace
`abs`	17	Convierte una URL relativa en absoluta.
`MediaMetadata`	19	Reemplazo de MediaMetadata para la página.
`bestArt`	28	Elige la carátula más adecuada (≤ 512 px) para la notificación.
`mediaSession.metadata`	42	Manda título, artista, álbum y carátula a Android.
`mediaSession.playbackState`	47	Manda el estado de reproducción a Android.
`mediaSession.setActionHandler`	54	Registra los botones que maneja la página.
`mediaSession.setPositionState`	58	Manda la posición solo cuando se desvía.
`__nanuqMedia`	76	Ejecuta en la página un botón de la notificación o de los audífonos.
`sendTheme`	87	Manda el color del tema a las barras del sistema.

`android/build-apk.sh` 19 líneas · 1 funciones

Función	Línea	Qué hace
`(script)`	1	Compila el APK y lo deja en dist/<versión>/.

`android/app/src/main/java/com/nanuq/app/OggCheck.java` 76 líneas · 2 funciones

Función	Línea	Qué hace
`OggCheck.valid`	42	Revisa que un Ogg (Opus) tenga todas sus páginas completas, con CRC correcto y fin del flujo.
`OggCheck.mime`	70	Acepta el tipo de audio del servidor solo con la forma audio/....

19 Escritorio

Electron y su empaquetado.



linux/main.js

855 líneas · 61 funciones

Función	Línea	Qué hace
serve	47	Servidor interno 127.0.0.1:47863 que entrega la app a la ventana (y la fiesta), endurecido.
broadcastOf	79	Dirección de difusión de una red IPv4.
discover	80	Busca servidores Jellyfin por UDP 7359.
fromApp	100	¿El mensaje IPC viene de la página de la app?
vaultBackend	132	Qué llavero del sistema hay (GNOME Keyring, KWallet, DPAPI, Llavero).
secFile	148	Ruta del archivo de datos cifrado.
secKeyFile	149	Ruta de la clave de datos cifrada con el llavero.
writeAtomic	150	Escribe un archivo de forma atómica (0600).
secInit	153	Abre el almacén cifrado (AES-256-GCM): locked, lost o listo.
secFlush	190	Escribe el almacén cifrado en el disco.
dcSet	228	Anota el estado de la conexión con Discord.
dcPaths	229	Sockets o tuberías de Discord posibles (solo los tuyos).
dcWrite	240	Manda un mensaje con el formato de Discord.
dcRetry	241	Vuelve a buscar Discord en 10 s.
dcConnect	242	Se conecta a Discord probando cada socket y saluda.
dcData	258	Lee los mensajes de Discord con topes (READY, ERROR, cierre, ping).
dcFlush	281	Manda la actividad respetando el ritmo de Discord (y la mínima si la rechazó).
dcActivity	292	Arma la actividad «Escuchando» limpia y acotada.
jparse	329	Lee JSON de afuera sin toString/valueOf/__proto__.
ptStr	330	Convierte a texto seguro y acotado (sin controles).
ptIp	331	Dirección IP sin el prefijo ::ffff:.
ptLocal	335	¿La dirección es de tu red local?
ptOpenNow	342	¿La sala está abierta?

Función	Línea	Qué hace
`ptEmit`	343	Avisa a la página un evento de la sala.
`ptBase`	348	Carpeta en disco donde se guardan las canciones de la sala (en Linux, ~/.cache/nanuq; antes era la temporal, que en el Flatpak es RAM).
`ptCleanTemp`	353	Borra carpetas temporales de salas anteriores.
`ptNonce`	366	Nonce de AES-GCM de un bloque: 8 bytes al azar de la canción + el número de bloque.
`ptSeal`	367	Cifra y autentica un bloque de 64 KB de una canción de la fiesta con la clave de la sala.
`ptEncSize`	372	Tamaño exacto que debe tener el archivo cifrado (detecta archivos cortados o alargados).
`ptOpenRange`	373	Lee, comprueba y descifra solo los bloques de un rango pedido; falla si un bloque fue alterado.
`ptSniff`	389	¿Es audio? (por sus primeros bytes).
`ptCover`	402	Valida la carátula que manda un teléfono (JPEG/PNG/WebP ≤ 150 KB).
`ptPublic`	411	Datos públicos de una canción de la lista.
`ptGuests`	415	Invitados presentes y ausentes.
`ptAddrs`	416	Direcciones IPv4 de esta computadora.
`ptStatus`	421	Estado completo de la sala para la página.
`ptState`	427	Avisa el estado a todos (agrupado en 150 ms).
`ptStateNow`	428	Manda el estado ya.
`ptKill`	434	Quita una canción de la lista y del disco.
`ptFrame`	443	Escribe un marco WebSocket.
`ptSend`	450	Manda un mensaje JSON a un teléfono.
`ptDrop`	451	Cierra la conexión de un teléfono (con motivo).
`ptGone`	459	Un teléfono se desconectó (queda ausente o salió).
`ptUpgrade`	470	Acepta la conexión WebSocket (solo red local, topes).
`ptData`	490	Lee marcos WebSocket con todas las validaciones.
`ptMsg`	523	Atiende un mensaje de un teléfono.
`ptNewCode`	536	Cambia el código de la sala.
`ptHello`	540	Entrada de un teléfono: código, llave, fuerza bruta.
`ptAdd`	570	Un teléfono anuncia una canción (con topes).
`ptChunk`	588	Recibe un pedazo de la canción y lo escribe al disco.
`ptEnd`	606	La canción terminó de llegar: comprueba que sea audio.
`ptServe`	623	Sirve canciones y carátulas de la sala a la ventana (con Range).

Función	Línea	Qué hace
ptGuestCover	651	Sirve carátulas a los teléfonos (solo con su llave).
ptGc	660	Limpieza cada 30 s (viejas, a medias, ausentes, mudas).
ptListen	670	Abre el puerto de la sala (47864 o uno libre).
ptOpen	683	Abre la sala: servidor, UDP, código y token.
ptClose	710	Cierra la sala y corta todas las conexiones.
stateFile	770	Archivo con el tamaño y posición de la ventana.
loadState	772	Lee el tamaño y posición de la ventana.
saveState	777	Guarda el tamaño y posición de la ventana.
start	783	Arranca la app: almacén, servidor, ventana, permisos y protecciones.

linux/preload.js

25 líneas · 6 funciones

Función	Línea	Qué hace
nanuqDesktop.discover	7	Puente: buscar Jellyfin en la red.
nanuqDesktop.http	8	Puente: petición sin CORS a SomeDL.
nanuqDesktop.discord	11	Puente: actividad de Discord (o su estado).
nanuqDesktop.party.op	14	Puente: operaciones de la fiesta (abrir, cerrar, saltar, ordenar, karaoke…).
nanuqDesktop.party.on	15	Puente: escuchar los eventos de la sala.
nanuqDesktop.secure.*	17	Puente síncrono al almacén cifrado (kind, get, set, del, reset).

linux/prepare-web.js

16 líneas · 1 funciones

Función	Línea	Qué hace
(script)	3	Copia web/ a www/ y toma la versión de app.js.

linux/collect.js

15 líneas · 1 funciones

Función	Línea	Qué hace
(script)	3	Mueve los paquetes compilados a dist/<versión>/.

linux/flatpak/build-flatpak.sh

54 líneas · 1 funciones

Función	Línea	Qué hace
(script)	1	Arma el Flatpak, lo publica en el repositorio local firmado y lo instala o actualiza.

20 Compilación

El script que compila todo.



scripts/build.sh

68 líneas · 1 funciones

Función	Línea	Qué hace
(script)	1	Sube la versión y compila Android, Linux, Flatpak y Windows.

Hecho con cariño (y mucho hielo)

Si cambias algo, recuerda: la interfaz vive en web/ y cada texto pasa por tx().

